Master Collaborative Reference: Reconstructing HELM Evaluations

Internship Chronology, Evidence Audit, and Publication Roadmap

Kitware Research Internship — Summer 2026

Edward Wang
edward.wang@kitware.com
Kitware, Inc.

Evidence cutoff: 2026-07-14; collaborative interpretation revised 2026-07-15; consensus and evidence-packaging pass 2026-07-15c

Abstract

This master reference consolidates two independently reconstructed chronologies, the weekly internship decks, developer journals, Git history, runbooks, paper sources, and technical design documents for Edward Wang's Kitware internship from May 15 through July 14, 2026. The initial goal—large-scale reproduction of public HELM results—became a study of a deeper measurement problem: a public benchmark recipe does not necessarily identify the complete experiment that produced a score. The surviving record may omit model and tokenizer revisions, load precision, chat-template behavior, serving engine and kernel details, hardware-sensitive numerical settings, harness era, and artifact-migration history. The internship built exact `run_spec.json` replay, containerized and era-pinned execution, GPU leasing, layered comparison reports, and deployment-matching tools to attribute disagreement across these layers. The central OLMo float32 result is reported with its correct status: a 12-instance IFEval discovery probe in which float32 candidates best matched sampled public outputs, supported by a conditional source-level argument about the pre-v5 Transformers dtype default, but not yet confirmed through the ordinary end-to-end HELM path on held-out data. The historic classic-model investigation supplies the complementary result: some provenance is recoverable, whereas migrated class paths and unreleased producing code can make the original measurement non-identifiable from the surviving evidence examined. This document distinguishes inherited pre-internship EEE work from internship contributions, preserves contemporaneous interpretations without presenting superseded claims as final conclusions, and provides an evidence-status ledger and a bounded publication roadmap.

Contents

1 Evidence Status and Paper Lineage **4**
- 1.1 Consensus rulings from independent reconstructions . . . 4
- 1.2 Evidence precedence . . . 6
- 1.3 Evidence-access asymmetry and attestation . . . 6
- 1.4 Canonical claim ledger . . . 7
- 1.5 Two-paper lineage . . . 7
- 1.6 Three orthogonal reproduction targets . . . 7
- 1.7 Corrections applied in this master revision (2026-07-15c) . . . 7

2 How to Read This Document **9**
- 2.1 Scope and provenance . . . 9
- 2.2 A note on inherited foundations versus internship contributions . . . 9
- 2.3 The two threads . . . 10

3 The Research Question and Its Framing **11**
- 3.1 From "reproduce HELM" to a falsifiable claim . . . 11
- 3.2 The load-bearing distinction: eligibility and disagreement layers . . . 12
- 3.3 The inherited baseline: what was already known . . . 12

4 Onboarding and Literature Review (mid-May – June 2) **14**

5 End-to-End Tests: Validating the Audit Instrument **15**
- 5.1 Motivation . . . 15
- 5.2 The controlled experiment . . . 15
- 5.3 Three instrument bugs found and fixed . . . 15
- 5.4 Outcome and next steps recorded at the time . . . 16

6 Corpus Cartography: Mapping the Reproducible Subset **17**
- 6.1 The question: what *can* be run at all? . . . 17
- 6.2 The numbers . . . 17
- 6.3 Why this is Stage 1 . . . 18

7 The Unified Comparison Core **20**
- 7.1 The refactor (Phases 0–2): making the instrument legible . . . 20
- 7.2 Phase 3 / `NormalizedDiff`: one core, many adapters . . . 20
- 7.3 The framework-free taxonomy and recipe-facts resolver . . . 21
- 7.4 The open-judge extension (R2) . . . 21
- 7.5 Why the entry points stayed separate . . . 22

8 The OLMo Campaign **23**
8.1 The target grid . . . 23
8.2 Chat-template overhead blows the context window . . . 24
8.3 Data-access failures are filtering reasons, not results . . . 24
8.4 The canonical-key matching bug: 114 phantom misses . . . 24
8.5 The BBQ prompt drift: a genuine recipe disagreement . . . 25

9 Containerization and GPU Leasing **26**
9.1 Why containerize: the environment is a variable . . . 26
9.2 From profiles to a leasing catalog . . . 27
9.3 The leasing pipeline: preconditions as job phases . . . 27

10 Faithful Replay: From Run-Entry to Run-Spec **29**
10.1 The core methodological principle . . . 29
10.2 Two failure modes, one loud and one silent . . . 29
10.3 Making the deployment substitution visible . . . 30
10.4 Addressing runs by (root, relative-path) and content-addressed caching . . . 30

11 Deployment Matching: The Flagship Investigation **32**
11.1 The provocation: OLMo-7B produces prompt-independent gibberish . . . 32
11.2 The tool: sweep, don't guess . . . 33
11.3 Chat-template version drift: `add_generation_prompt` . . . 33
11.4 The float32 discovery: a flagship attribution pilot . . . 33
11.5 The residual puzzle: unresolved Hugging Face divergence for dense OLMo-2 . . . 35
11.6 Naming the gap: the unrecorded execution substrate . . . 36

12 Era-Pinned Containers: Reproducing the Pre-v0.5 Corpus **37**
12.1 The problem: the measurement instrument itself has versions . . . 37
12.2 The design invariants . . . 37
12.3 The v0.2.4-versus-v0.3.0 API skew . . . 38
12.4 A turnkey validation runbook and the 8 GB subject . . . 38
12.5 G13: a class path that resolves in no released HELM . . . 38

13 New Model Families: GPT-OSS and Qwen **40**
13.1 GPT-OSS-20B and the null-content crash . . . 40
13.2 Qwen: generalizing the combined fan-out . . . 41
13.3 The Qwen turbo window bug . . . 41

14 Reporting and Analysis Infrastructure **43**
14.1 The pipeline the reports run on . . . 43
14.1.1 The three-level coverage funnel . . . 43
14.1.2 Sankey diagrams and the failure taxonomy . . . 44
14.2 The aggregate score-drift heatmaps . . . 44
14.3 Correctness fixes that changed the numbers . . . 44
14.4 The shared-gateway route registry . . . 45

15 Codebase Stewardship **46**

16 Thread B: Efficient Benchmarking **47**
16.1 The two framings being compared . . . 47
16.1.1 Feature selection + regression (Bowyer et al.) . . . 47
16.1.2 DKPS: a model-centric, transductive predictor . . . 47
16.2 Experiment design . . . 48
16.3 The DKPS+mRMR active-selection method . . . 48
16.4 Findings . . . 48
16.5 What stayed a proposal . . . 49

17 Synthesis: What Is Reproducible, and Why **50**
17.1 The headline scientific claims . . . 50
17.2 The reusable methodology . . . 51
17.3 The provenance recommendation for the field . . . 51
17.4 Per-parameter identifiability map . . . 52
17.5 Open threads at the close of this chronology . . . 52

A The HELM Gotchas Catalogue (G1–G13) **54**

B The Unrecorded-Parameter Taxonomy **56**

C Deliverables and Timeline **58**
C.1 Internship timeline at a glance . . . 58
C.2 Key artifacts produced . . . 58
C.3 A note on method and provenance of this document . . . 59

D Interpretive Revision (2026-07-15): Epistemic Status of the Central Claims **60**
D.1 A workflow-status vocabulary . . . 60
D.2 A finer disagreement taxonomy . . . 60
D.3 Corrected-claims table . . . 61

E Collaborative Claim–Evidence–Status Matrix **63**

F Source Inventory and Reproducibility Notes **65**

G Publication-Grade Preservation Checklist **66**

H Main-repository commit ledger **67**

I Submodule contribution ledger **83**

Chapter 1

Evidence Status and Paper Lineage

This edition reconciles two independent reconstructions of the internship record. They agree on the chronology and engineering substance. The main differences concerned the strength of empirical claims, the distinction between a deployment-matching probe and an authoritative benchmark reproduction, and the evidentiary status of live result stores omitted from the review archive. The following consensus rulings govern all later chapters; where an older passage records a stronger contemporaneous belief, this chapter and the inline status callout take precedence.

1.1 Consensus rulings from independent reconstructions

Point	Ruling	Reason and qualification
The EEE paper and the internship follow-on are distinct paper lines.	**Established**	The NeurIPS case study and 2,626-line technical report predate the internship and focus on normalization and discrepancy detection. A separate TMLR skeleton existed on May 14. The follow-on should be positioned as attribution, possible closure, and identifiability rather than a repetition of EEE's Pythia/Vicuna/Falcon study.
The positive thesis "most residual disagreement is small and attributable" should be demoted.	**Established**	It is a motivating hypothesis, not a result, until the denominator is frozen, report stores are fresh, selected configurations are confirmed through the normal HELM path, discovery is separated from held-out evaluation, and repeatability/ablation experiments are complete.
The OLMo float32 result is discovery rather than completed reproduction.	**Established**	The ranking comes from a small oracle-scored IFEval probe. The winning request behavior may include `add_special_tokens=False`, which the ordinary HELM path does not send automatically. Authoritative confirmation requires an exact-spec local run, normal HELM artifacts, the standard pair report, and held-out evaluation.

Point	Ruling	Reason and qualification
The dtype argument has a deductive source-level leg.	**Preliminary**	Public OLMo instruct deployments are recorded as `HuggingFaceClient` with no pinned `torch_dtype`; the client did not inject one, and Transformers 4.x defaulted such loads to fp32. The January 2025 OLMoE timing and architecture bound the likely version to the 4.x regime. Because the exact historical environment is absent, the correct conclusion is "most likely fp32 under the bounded loader assumptions," not a uniquely proven full deployment.
Identifiability is per parameter rather than all-or-nothing.	**Established**	Client class and configured dtype presence are source-readable; tokenizer behavior and templates may be repository-readable; exact package builds, kernels, batch composition, hardware, and hosted-provider internals may remain unidentified. A per-parameter identifiability map is therefore a useful paper contribution.
The dense OLMo-2 Hugging Face divergence is scientifically relevant.	**Established**	Prompt-token equivalence rejects one class of explanations. The remaining divergence between the current HF stack and a result recorded as a `HuggingFaceClient` deployment is evidence of execution-stack sensitivity. Its cause remains unresolved — with a dense-model probe artifact still among the candidates — and must not be dismissed as merely a broken probe.
The six-category disagreement taxonomy is preferable to "true reproducibility failure."	**Established**	Non-runnable cases are an upstream eligibility gate. Observed disagreements should be classified as recipe, deployment, execution-instrument, or artifact-interpretation mismatch; residual disagreement under controlled equivalence; or historically non-identifiable reproduction.
Recoverable versus non-identifiable provenance is the strongest conceptual spine.	**Established**	OLMo supplies mechanisms whose missing parameters can be partly reconstructed. GPT-J/GPT-NeoX/OPT artifacts point to an unreleased producing instrument and cannot be uniquely mapped to a released harness. OLMo should be called *partially recoverable* until ordinary-path confirmation is complete.

Point	Ruling	Reason and qualification
RedPajama-3B ran end-to-end under both proxy eras.	**Collaborator-verified; artifact not packaged**	July 14 journal entries refer to intact era-RedPajama runs and refreshable report artifacts, and the slide deck states that the E2E script works for v0.2.4 and v0.3.0. The provided source archive explicitly excludes `/data`, so the reports themselves are not in this evidence package. Preserve and hash them before paper submission. Its "recovery" is a forensic proxy comparison against a public-side serving artifact, not proof that either later era reproduces the original producing instrument.
Preserving the dated narrative while appending corrections is acceptable.	**Implemented**	It preserves research history, but an authoritative reference must prevent readers from mistaking superseded wording for the final conclusion. This master edition therefore uses corrected wording inline and retains historical interpretations only as explicitly labeled contemporaneous beliefs.
The claimed commit `86ec84af` contains the revisions.	**Collaborator-verified; artifact not packaged**	A collaborating live-repository review reports that `86ec84af` was the current HEAD of `impl/run-from-run-spec` on 2026-07-15 and contained the stated revisions. The supplied review archive ends at `b8b8586`; the later Git object or bundle is not included in this package. Treat the fact as collaborator-verified until a Git bundle or immutable repository reference is preserved with the paper artifact.

1.2 Evidence precedence

For this master reference, evidence is prioritized as follows: immutable public or local artifacts with hashes; source code and Git history; generated reports whose inputs remain available; journals and runbooks; slide-deck summaries; and finally retrospective narrative interpretation. A report-store claim whose raw directory is excluded from the archive is not silently promoted to a packaged result.

1.3 Evidence-access asymmetry and attestation

The collaborating reviewers did not have identical evidence access. One review used the supplied `git archive`, which excludes `/data`, ignored files, and post-archive commits. A later review reports direct access to the live branch and those stores. This resolves several factual questions, but it does not replace an archival artifact. Claims based only on that access are marked **Collaborator-verified; artifact not packaged** until the relevant Git objects, manifests, report inputs, and hashes are copied into the release bundle.

For result stores, three independent freshness questions must be recorded:

1. **run-artifact freshness**: were the model outputs generated by the intended code, model revision, and execution path?;
2. **comparison freshness**: were those outputs compared with the intended public target using the corrected matching and deduplication logic?; and
3. **report freshness**: were tables and plots regenerated after the latest reporting fixes?

A recent report timestamp establishes only the third item unless the first two are also demonstrated by a manifest.

1.4 Canonical claim ledger

The machine-readable file `chronicle/data/master_claim_evidence_ledger_2026-07-15c.csv` is the canonical claim-status source for this reference set. Prose tables are readable snapshots and should be regenerated from, or checked against, that ledger before any submission. This prevents the chronology, strategy, and review appendices from drifting into incompatible statuses.

1.5 Two-paper lineage

The inherited EEE paper demonstrates that a normalized result representation can expose reproducibility gaps, but explicitly cannot separate checkpoint, quantization, precision, tokenizer, and generation-kernel effects. The follow-on paper should use the positioning sentence:

> *EEE detects reproducibility gaps; this work studies their attribution, possible closure, and identifiability.*

1.6 Three orthogonal reproduction targets

The six-category mismatch taxonomy answers *why* two executions disagree. A separate three-level axis answers *what* is being reproduced:

1. **Artifact reconstruction**: recompute the published aggregates from the released prompts, completions, and metric records without rerunning a model.
2. **Procedural reproduction**: execute the recorded recipe under a pinned or explicitly equivalent instrument and compare instance-level outputs and metrics.
3. **Claim-level robustness**: determine whether the substantive conclusion (for example, a ranking or model-generation improvement) survives plausible execution substrates even when byte-identical outputs do not.

These levels should not be collapsed. Released artifacts are the historical ground truth for what HELM reported; reruns assess the stability and identifiability of the procedure that produced them.

1.7 Corrections applied in this master revision (2026-07-15c)

Following a collaborating review of the *live* repository (reported HEAD `86ec84af` on `impl/run-from-run-spec`, including `/data` stores that the source archive omits), the following corrections were applied *inline*. Because the Git objects and raw stores are not themselves included, live-only facts are recorded as collaborator-verified rather than promoted to immutable packaged evidence. The reasoning and machine-readable attestations are preserved under `validation/claude_patch/`.

- **Commit** `86ec84af` reclassified from unresolved to **Collaborator-verified; artifact not packaged** — reported as the live branch head, pending inclusion of the Git object or an immutable remote reference.
- **Store status split three ways**, replacing the single "not packaged" label: the GPT-OSS *reports* are reported fresh but still need run/comparison acceptance plus preservation; `olmo-models-combined` is reported stale (on-disk `olmo-7b` halving `0.295/0.144`) and must be regenerated; and the Qwen store is reported genuinely absent (Chapter 13, Appendix D).
- **OLMo-2 HF divergence** now keeps a dense-model probe artifact on the live candidate-cause list, alongside genuine execution-instrument sensitivity (Chapter 11).
- **Wording and sourcing:** "apparently HF-produced" → "recorded as a `HuggingFaceClient` deployment"; the GPT-OSS-vs-OLMo `ifeval` drift ratio "two orders of magnitude" → "≈30×"; the 59% classic-track figure now carries its source.

The structural concerns from that review are also resolved in this consensus package: the claim ledger is canonicalized, vendor-specific collaboration meta is neutralized, and the full directory layout includes the referenced appendices and an accurate README.

Chapter 2

How to Read This Document

2.1 Scope and provenance

This is an *authoritative reference* assembled after the fact from primary sources so that a future paper can cite it with confidence. The reconstruction draws on:

- the seven weekly intern status decks (2026-06-02 through 2026-07-14), which record the work in the author's own words and contain the headline result figures;
- the append-only developer journals (`dev/journals/claude.md` and its `2026-H1` archive), which capture, per work session, the intent, the design alternatives considered, the chosen approach, and the reusable lesson — roughly one hundred design narratives;
- the git history: 452 commits authored between 2026-06-03 and 2026-07-14, whose messages give a fine-grained, dependency-ordered record of what landed and when;
- the standing research journal (`docs/helm-reproduction-research-journal.md`), which frames the scientific question and records the early foundational findings; and
- the technical design documents accumulated under `docs/`, chiefly `helm-gotchas.md`, `vllm-vs-huggingface-de` `helm-unrecorded-deployment-params.md`, `eee-vs-helm-metadata.md`, `pipeline.md`, `architecture.md`, and `container-execution.md`.

The internship ran from 2026-05-15. The first ∼2.5 weeks (through the 06-02 status update) were spent on onboarding and a literature review; the first authored commits appear on 2026-06-03. The work then proceeds essentially without interruption through 2026-07-14, the last date this chronology covers.

2.2 A note on inherited foundations versus internship contributions

The internship did not start from an empty repository. A substantial foundation was already in place: the `eval_audit` package itself; the *Every Eval Ever* (EEE) normalized artifact format used as the common comparison substrate; the `kwdagger` scheduler integration; and an early set of reproducibility findings on BOOLQ/Pythia and Vicuna/NarrativeQA (described in §3.3). Where this document describes that foundation, it does so to make the internship's own contributions legible; the chronology proper (Chapters 5 onward) is the author's work. The developer journals are written in the first person by the AI pair-programming harness (Claude Code) that the author drove; throughout this report, "we" denotes the author directing that toolchain, making the design decisions, and executing and interpreting the runs on the GPU hosts.

2.3 The two threads

Two research threads ran in parallel for the first month, after which the second was paused in favor of the first:

Thread A — HELM reproduction (Chapters 3–14).
The main line of work: building the machinery to faithfully replay public HELM runs on local open-weight hardware, and using it to determine what is and is not reproducible, and why. This is the bulk of the internship and of this document.

Thread B — efficient benchmarking (Chapter 16).
An exploratory statistics thread: can a small, well-chosen *coreset* of benchmark questions plus a regression predict a model's full-benchmark score, and does a Data-Kernel Perspective-Space (DKPS) representation help? This lived in two sibling repositories (`mrmr_eval`, `dkps`) and reached a working prototype with preliminary results before being paused.

Chapter 3

The Research Question and Its Framing

3.1 From "reproduce HELM" to a falsifiable claim

The stated initial goal was to perform large-scale reproductions of the public HELM (Holistic Evaluation of Language Models) results. The public corpus publishes, for many (scenario, model) pairs, the prompts sent, the completions returned, and the computed metrics. Reproducing a result means re-running the same evaluation locally and checking that the numbers match.

That goal did not survive contact with the data, and the way it failed is itself the scientific content. Two categories of models dominate the public corpus: open-weight models that were originally served through a hosted API (frequently *Together AI*), and closed-weight models that cannot be run locally at all. The recipe that produced each public number is only partially recorded: the `run_spec.json` captures *what* to generate (prompts, decoding parameters, scenario, metrics) and the *name* of the model and its deployment, but almost nothing about the *execution substrate* — the precision the weights were loaded at, the exact checkpoint revision, the attention kernel, the tokenizer and chat-template versions, the software-stack versions. As a result, three distinct difficulties compound:

1. **Running the benchmarks at all** — gated models and datasets, packaging incompatibilities, download endpoints that have since revoked access, and scenario code that requires optional dependencies or credentials.
2. **Setting up a *faithful* local reproduction** — matching the original recipe when key execution parameters were never written down, and when the original was served by an external provider whose stack we cannot inspect.
3. **Attributing disagreement** — when local and public numbers differ, deciding whether the benchmark is legitimately non-reproducible, whether the difference is an artifact of an uncontrollable external deployment, or whether our local reproduction is simply missing a detail.

The goal was therefore reframed. The paper-worthy claim is *not* "all of public HELM is reproducible." It is:

> *A public benchmark recipe is not a complete experimental record. Reproduction is a layered system-identification problem: controlled reconstruction can attribute and sometimes close apparent gaps, while lost or transformed provenance can leave the original measurement non-identifiable.*

3.2 The load-bearing distinction: eligibility and disagreement layers

The first gate is whether a run is executable at all. Gated models or datasets, revoked download access, missing credentials, unsupported packaging, unavailable judges, and infeasible resource requirements are *eligibility or environment filters*; they are not negative reproduction measurements.

For runs that execute and can be compared, the final taxonomy is:

1. **Recipe mismatch**: scenario, prompt construction, examples, decoding, or metrics differ.
2. **Deployment mismatch**: the client, hosted endpoint, serving engine, or exposed deployment identity differs.
3. **Execution-instrument mismatch**: model/tokenizer revision, dtype, quantization, template implementation, software stack, kernels, topology, batching, caching, or hardware-sensitive numerical behavior differs.
4. **Artifact-interpretation or migration mismatch**: stored artifacts were reordered, normalized, carried forward, or migrated under later schemas.
5. **Residual disagreement under controlled equivalence**: material controllable layers are matched and disagreement remains. This is the only category analogous to conventional repeatability failure.
6. **Historically non-identifiable reproduction**: surviving evidence does not uniquely specify what execution should count as faithful.

The project's earlier binary—environment/recipe failure versus "true reproducibility failure"—was a useful operational first cut, but it is preserved only as a contemporaneous framing. The six-layer taxonomy is the authoritative one.

3.3 The inherited baseline: what was already known

Before the internship, the project had established a small but load-bearing set of results, recorded in `docs/helm-reproduction-research-journal.md`. They set the priors that the internship's work refined.

- **Local reruns are stable; the official–local gap is not noise.** For `boolq:model=eleutherai/pythia-6.9b`, two local repeats agreed at a strict run-level ratio of 0.955 with a maximum run-level delta of 0.0015, whereas official-vs-local agreement was 0.463 with run-level deltas up to ~ 12. The official–local gap is far larger than rerun noise, so ordinary nondeterminism cannot be the main explanation — the residual is structural (deployment and execution-spec drift).
- **A dramatic "irreproducibility" turned out to be a local misconfiguration.** Local Vicuna/NarrativeQA produced 99/100 empty completions; the cause was HELM silently auto-applying a *chat template* to a non-chat prompt. Rerunning with `apply_chat_template: false` recovered near-official scores (`exact_match` 0.27, `f1` 0.64, `rouge_l` 0.64). The lesson — that `apply_chat_template` must be an explicit, controlled setting, and that high empty-completion rate / near-zero output tokens / pervasive `finish_reason_unknown` are red flags — recurs throughout the internship.
- **The metric-scale caveat.** A single global tolerance is hard to interpret because *core* metrics live in $[0, 1]$ while *bookkeeping* metrics (byte counts, token counts, log-probs) do not. The reporting must separate metric classes; this became the `metrics_taxonomy` module (§7.3).

These priors motivated the internship's dominant methodological choice — prefer open-weight models runnable on realistic hardware, control the deployment explicitly rather than trusting

HELM's automatic inference, and treat the reproducibility question as a *distribution* (same-machine, cross-machine, official-vs-local) rather than a single point estimate.

Chapter 4

Onboarding and Literature Review (mid-May – June 2)

The first status deck (2026-06-02) reports two workstreams: end-to-end tests for the audit pipeline (Chapter 5) and a literature review that seeded both the reproducibility thread and the efficient-benchmarking thread (Chapter 16). Three papers were reviewed in depth.

LoRA weight-space learning — *W2T: LoRA Weights Already Know What They Can Do* (Han et al., 2026), building on Chen, Helm, Park, Priebe & Villar, *Extracting information from fine-tuned weights*.
A LoRA update is $AB^\top$; the factorization (A, B) is non-identifiable because $(AM, BM^{-\top})$ yields the same update for any invertible M (a GL-group symmetry). W2T canonicalizes via the SVD and embeds the canonicalized weights with a transformer into a "weight-space," whose embeddings drive downstream tasks: attribute classification (properties of the fine-tuning data), performance prediction, and adapter retrieval. The review noted that the ablations attribute most of the method's power to the *canonicalization* step, and floated directions: generative modelling over LoRA weights, simpler statistical embeddings (e.g. MDS) in place of the transformer, and multi-/merged-adapter settings.

Efficient benchmarking — *Efficient Benchmarking Is Just Feature Selection and Multiple Regression* (Bowyer, Locatelli & Cao, 2026).
Given M source models, represent each question X_i as a vector in $\mathbb{R}^M$ (each coordinate is a model's score on that question) and the target Y as the vector of overall scores; *select* an informative subset of questions with mRMR (Minimum-Redundancy-Maximum-Relevance) and *predict* full-benchmark scores with ridge / kernel-ridge regression. Baselines: Lasso, Anchor Points, IRT. This framing became Thread B.

Connecting the two.
The review proposed comparing LoRA and DKPS (Data-Kernel Perspective-Space) representations, using Bayesian / Gaussian- process regression to unify selection and regression, and enriching the per-question feature vector with question-text embeddings — directions pursued in Chapter 16.

Chapter 5

End-to-End Tests: Validating the Audit Instrument

Commits: 2026-06-03 through 2026-06-04, and the E2E deck of 2026-06-02.

5.1 Motivation

Before trusting the audit pipeline's verdicts, the pipeline itself had to be validated end to end on a case where the ground truth was controllable. The audit workflow has six stages: (1) stand up a vLLM server for the target model (via `infer-stack`, over LiteLLM / docker-compose); (2) write a benchmark bundle; (3) execute runs on the servers; (4) aggregate run artifacts; (5) index them; (6) build the analysis. The tools involved — `infer-stack`, `eval-audit`, `magnet`, `kwdagger`, `cmd_queue` — each have seams where a subtle mismatch produces a plausible-but-wrong comparison.

5.2 The controlled experiment

The subject was `microsoft/phi-2` on `mmlu:philosophy`, run under three deliberately chosen deployments:

- **HF**: HELM's in-process `HuggingFaceClient`, `temperature=0`;
- **vLLM**: served through vLLM, `temperature=0` — should match HF up to engine numerics;
- **vLLM "incomparable"**: served through vLLM at `temperature=1` — a deliberate negative control that *should* disagree, proving the instrument can detect a real difference.

5.3 Three instrument bugs found and fixed

Validating the instrument surfaced three ways the comparison machinery could silently mispair runs — each fixed so that equivalent runs are actually recognized as equivalent:

1. **Hard-coded serving ports.** `infer-stack` used hard-coded ports without exposing them in a form HELM could consume. Fix: read ports from the configuration source of truth and export them into a `.env` file queryable by the rest of the pipeline.

2. **The `-groups` flag.** Public benchmark runs carry a `-groups` flag that has no effect on execution but prevented equivalent local runs from being matched to their public counterparts. Fix:

ignore `groups=` tokens when locating comparable public runs, and emit a warning that this normalization occurred.

3. **The `-temperature` run-name elision.** HELM strips the temperature from the run *name*, creating a divergence between the run spec and the run-directory name, so temperature-varying runs could not be located. Fix: locate run folders more robustly by reading the original run spec out of the generated artifacts rather than trusting the directory name.

These three fixes are small but foundational: they are the first instances of a theme that dominates the rest of the internship — HELM's run identity is carried by a drift-prone *string*, and robust reproduction requires matching on *canonicalized recipe content*, not names (revisited at scale in §8.4 and in gotchas G1–G2, G10–G12; see Appendix A).

5.4 Outcome and next steps recorded at the time

The e2e tests established that HF and vLLM at `temperature=0` agree on the core metric while the `temperature=1` control diverges as intended — i.e. the instrument both confirms equivalence and detects real differences. The deck's next steps (automate the e2e suite into CI, improve the warnings the systems emit, and begin auditing more public models) foreshadow the rest of the summer.

Chapter 6

Corpus Cartography: Mapping the Reproducible Subset

Status deck of 2026-06-09.

6.1 The question: what *can* be run at all?

Before reproducing anything, one must know the shape of the corpus and which slice is even a candidate. A statistics-and-filtering script was written to crawl a directory of HELM runs, extract per-run metadata, and emit a machine- readable filter inventory (`filter_inventory.json`). For each run it records the model (parameter count, access class {open, limited, closed}), the scenario and its class path, the benchmark family, whether the task requires a closed (proprietary) judge, and a *selection status* with explicit *failure reasons*. A representative row (a Thai-exam Typhoon-8B run) carries fields such as `model_num_parameters`, `model_access`, `model_has_hf_client`, `size_threshold_params`, `eligible_model`, `candidate_pool`, and a `failure_reason_summary` — the vocabulary that the Stage-1 Sankey diagrams later consume.

6.2 The numbers

The crawl over the public corpus produced a stark funnel (Table 6.1). Of **34,512** total runs, only **1,109** were *selected* as candidates for local reproduction under the open-weight, runnable-recipe policy. The model-access distribution alone explains much of the attrition: nearly as many *limited*-access (18,273) as *open* (12,823) runs, plus 508 closed and 2,894 with no recorded access class. Parameter counts were available for only 13,979 runs, so a size budget could be applied to fewer than half the corpus with certainty.

Quantity	Count
Total runs discovered	34,512
Structurally incomplete	14
Eligible model, out of scope	5
Selected (candidates)	**1,109**
Model access: open	12,823
Model access: limited	18,273
Model access: closed	508
Model access: none recorded	2,894
Runs with parameter-count data	13,979
Full grid points (models × scenarios)	78,690
Full grid: models / scenarios	366 / 215
Filtered grid points	2,013
Filtered grid: models / scenarios	33 / 61
Runs requiring a closed judge	484
Runs excluded *solely* for closed judge	25

Table 6.1: The public-HELM corpus funnel as measured on 2026-06-09. The runnable subset is roughly 3% of the corpus, and grid coverage is sparse: 198 of 2,013 filtered grid points are actually populated ("likely not all scenario/model pairs make sense").

Finding. The closed-judge dependency is real but not dominant.

Of 484 runs that require a closed judge, only 25 are excluded *solely* for that reason — the rest fail an independent filter too. This number motivated the later *open-judge extension* (§7.4): admitting closed-judge benchmarks behind a flag and re-running them with an open judge recovers a non-trivial-but-bounded slice.

Finding. Grid coverage is sparse and non-rectangular.

The full model×scenario grid has 78,690 points but only 6,844 are covered; the filtered grid has 2,013 points and 198 covered. This is the empirical basis for treating reproduction as a *coverage problem* — the denominator of any "fraction reproducible" claim must be the set of populated, in-scope grid points, not the Cartesian product. It directly motivates the three-level coverage funnel (logical / recipe-canonical / recipe-identical) that the pipeline reports (§14.1.1).

6.3 Why this is Stage 1

This cartography is Stage 1 of the pipeline (`index_historic_helm_runs.py`): it discovers runs, applies the eligibility filters, and emits the filter report whose Sankey diagram shows what was kept or dropped and *why*. Its output is the "denominator" the architecture document (ADR-6) insists must remain explainable: every excluded run carries its typed exclusion reason, so a reviewer can audit the claim "we reproduced X of Y runnable runs." Crucially, the exclusion reasons here are almost all *recipe/environment* reasons (§3.2) — gated model, closed judge, no local deployment, over the size budget — not reproducibility verdicts.

> **Corpus caveat (recorded later).** The curated candidate set that drives the reproduction runbooks (`configs/run_details.yaml`) is a hand-selected Pythia/Vicuna/Qwen/OLMo subset, not the full eligible list; and an on-disk `filter_inventory.json` captured at one moment can

go stale relative to the live corpus (e.g. a modern-only pass that contains zero classic-era rows). The distinction between "the corpus" and "the curated candidate set" matters when reading any coverage number.

Chapter 7

The Unified Comparison Core

Commits: 2026-06-11 through 2026-06-12 (roughly 30 commits); the "Phase 0–3" refactor and the "Phase 3 / 4.0–4.9" comparison-core unification.

Two days of intense work rebuilt the analysis substrate so that every subsequent reproducibility verdict flows through a single, framework-free comparison engine. This is infrastructure, but it is infrastructure with research consequences: it is what makes the later findings *auditable* rather than asserted.

7.1 The refactor (Phases 0–2): making the instrument legible

The repository had grown to ∼86 modules and ∼32.8k lines with 18 console scripts, and several "god modules" had accreted — notably `build_reports_summary.py` at 5,369 lines / 108 functions. The refactor proceeded in phases: root hygiene and finishing an in-flight `vllm_service`→`infer_stack` rename (Phase 0); unifying the CLI surface so every console script resolves to a thin `eval_audit.cli.*` wrapper (Phase 1); and decomposing the god modules into cohesive subpackages (Phase 2).

The methodological point worth recording for the paper's "reproducible software" story is *how* the splits were verified. Rather than hand-moving functions, the god modules were split programmatically: AST-parse the module, compute the top-level-symbol reference graph, validate the cluster assignment is a DAG, generate each submodule with exactly the imports its symbols reference, then assert that all top-level symbols are *AST-identical* to `HEAD`. This gives a machine-checkable "pure relocation" guarantee. A recurring subtlety — that monkeypatching a facade module no longer reaches functions that bind their collaborators in a new home module — caused several test fixes and is a lesson about characterization tests surviving refactors.

7.2 Phase 3 / `NormalizedDiff`: one core, many adapters

The deeper change was to unify two forked comparison cores. The HELM path computed agreement and diagnosis through `HelmRunDiff` (a 1,902-line class); the EEE path computed overlapping numbers through `normalized/compare.py`. Both were invoked from one function, `_build_pair`, gated by a `skip_diagnosis` flag. The agreement math was already framework-free on both paths; only the *diagnosis* block and the `run_spec.json` semantic diff were HELM-specific.

The unification introduced `eval_audit/normalized/diff.py::NormalizedDiff`: one framework-free core consuming two `NormalizedRun` objects and producing agreement rows, agreement curves, quantiles, the facts-grade diagnosis, and the judge-dependence metric-class split. The HELM and

EEE paths became *thin adapters* over this one core; `HelmRunDiff`'s `_diagnose_repro` was reduced to delegating to `normalized.diagnose.diagnose_repro`.

The design was executed *additive-first*, which is the load-bearing rigor: `NormalizedDiff` was built next to the incumbent and proven *byte-equal* to committed baseline snapshots (fixtures F1–F10, gated at numerical tolerance `atol`$= 10^{-9}$, with the explicit rule that a looser tolerance is "a finding to investigate, not a knob to widen") *before* any renderer was flipped to use it. Inverting the usual order — build the replacement, prove equivalence, then swap — turned a "high-risk hinge" into two routine commits.

7.3 The framework-free taxonomy and recipe-facts resolver

Three supporting modules were lifted out of the HELM-specific package so they carry no framework dependency:

- `eval_audit/metrics_taxonomy.py` — classifies every metric as *core* vs *bookkeeping* (the metric-scale caveat from §3.3) and, new here, as *judge-dependent* vs *deterministic* (`classify_judge_dependence`). The latter split lets a report separate the part of a comparison that is a *reproduction control* (deterministic metrics) from the part that is a *substitution measurement* (judge-dependent metrics).
- `eval_audit/normalized/recipe_facts.py` — one resolver answering "what recipe produced this run?" in priority order: a native `recipe_facts` block (JSON-encoded inside an EEE aggregate's `source_metadata.additional_details`) → a sidecar `run_spec.json` → *unknown* (facts collapse to `status='unknown'` and the pipeline emits `comparability_unknown:*` warnings). This is the seam through which the later "unrecorded parameters" story (Appendix B) could be closed without a schema change.
- `eval_audit/normalized/diagnose.py` — the single input-to-label diagnosis implementation, including declared-substitution re-labeling (`intended_substitution:*`).

7.4 The open-judge extension (R2)

A one-hour "spike" against the real corpus reshaped a planned feature and recorded a genuinely useful finding (`docs/planning/judge-identity-inventory.md`). The question was: where does judge-model identity live for the six closed-judge benchmarks? Three findings:

1. **Judge identity is not in `run_spec.json`.** Annotators carry *empty* args; the judge model is hard-coded inside the HELM annotator class as a function of HELM version (e.g. a GPT-4o + Llama-3.1-405B ensemble). Officials therefore need a curated (annotator class, version)→judge-models map, implemented as `eval_audit/judge_registry.py`.
2. **The official ensemble already contains an open judge.** Because HELM records per-judge sub-scores (e.g. `safety_gpt_score`, `safety_llama_score`) as separate metrics, a *same-judge reproduction control already exists inside the official data* — one can compare `safety_llama_score` official-vs-local without any new run. This reframes substitution as per-metric rather than per-run.
3. **Honesty scoping.** The `same_judge` comparability fact is emitted *only* for declared-substitution comparisons; emitting it everywhere would spray `comparability_unknown:same_judge` noise over every legacy pair. A crucial design decision recorded here: facts stay honest (`same_judge: no` is never re-labeled to a "substituted" status); only the *diagnosis* re-labels the difference as `intended_substitution:judge`. A "substituted" fact status was rejected as dishonest-by-construction.

Stage 1 gained `-allow-closed-judge-benchmarks`, admitting these runs through a distinct `judge-substitution` candidate pool that rides `run_details`→manifests→audit index to the planner.

7.5 Why the entry points stayed separate

A tempting simplification — collapse the HELM and EEE CLIs into one auto-detecting command — was deliberately *rejected*, and the reasoning is worth preserving because it recurs. The EEE-only path must remain physically capable of building its full report tree *with zero HELM artifacts on disk* (the "EEE-only operability" gate), because one of the paper's case-study claims is that the EEE per-instance schema alone suffices for reproducibility analysis — a claim a reviewer will want to verify by `grep`, not by trusting a code review. Merging the CLIs would import the HELM adapter into the EEE path and destroy that grep-checkable guarantee.

Chapter 8

The OLMo Campaign

Commits: 2026-06-12 onward; status decks of 06-16, 06-23, 06-30, 07-08, 07-14. The OLMo reproduction is the internship's central case study, and the vehicle through which every piece of reproduction machinery was hardened.

8.1 The target grid

Six open-weight OLMo-family models were chosen as the reproduction subjects (Table 8.1). Four are recent instruct models evaluated on the capabilities benchmarks (`bbq`, `gpqa`, `ifeval`, `mmlu_pro`); two are older base models with broad classic-track coverage. The grid spans both *no-chain-of-thought / temperature=0* benchmarks (bbq, commonsense, gsm, ifeval, legalbench, med_qa, mmlu, narrative_qa, wmt_14) and *chain-of-thought / temperature=1* benchmarks (gpqa, mmlu_pro).

Model	**Benchmarks reproduced**
`olmoe-1b-7b-0125-instruct`	bbq, gpqa, ifeval, mmlu_pro
`olmo-2-1124-7b-instruct`	bbq, gpqa, ifeval, mmlu_pro
`olmo-2-1124-13b-instruct`	bbq, gpqa, ifeval, mmlu_pro
`olmo-2-0325-32b-instruct`	bbq, gpqa, ifeval, mmlu_pro
`olmo-7b`	mmlu (62), math (7), legalbench (5), wmt_14 (5), natural_qa (2), commonsense (1), gsm (1), med_qa (1), narrative_qa (1)
`olmo-1.7-7b`	mmlu (57)

Table 8.1: The six OLMo reproduction subjects and their benchmark coverage. The instruct models exercise the chat-template and judge-dependent paths; the base models exercise the classic-track, completions path.

Early smoke tests already surfaced two environment issues that recur: a shared-machine configuration-file conflict, and `olmo-7b`'s tokenizer attempting to run external code (an early symptom of the EOS-append bug diagnosed in §11.1). These were fixed via HELM tokenizer-alias corrections, adding `sentencepiece`/`tiktoken` tokenizer backends as dependencies, and loading the OLMo-7B tokenizer through a HELM entry-point plugin.

8.2 Chat-template overhead blows the context window

The first substantive reproducibility bug in the OLMo runs was a `ContextWindowExceededError`: the chain-of-thought runs (`num_output_tokens=2048`) on the 4096-context chat models were rejected by vLLM. The mechanism is subtle and paper-worthy. HELM's `LocalWindowService` truncates the prompt to max_seq_len − expected_completion = 4096 − 2048 = 2048 tokens, but it counts the *raw* prompt, *before* the chat template is applied. When the run is routed through vLLM's OpenAI-compatible chat endpoint, the OLMo-2 / OLMoE chat template wraps the prompt and adds tokens HELM never counted. Measured with the actual tokenizers, the single-turn wrapper overhead is **12 tokens (OLMo-2)** and **13 tokens (OLMoE)**, so $2048 + 13 + 2048 = 4109 > 4096$ and vLLM hard-rejects.

The fix reserved a 32-token margin per preset via the sanctioned `helm_max_sequence_and_generated_tokens_le` knob (already used by the Vicuna chat path), and repaired a latent bug where the flat-preset code path silently dropped that override. The general lesson — *when HELM drives a chat model through an OpenAI-compatible server, its prompt-token budget is blind to the chat template the server applies* — is the first of several chat-template-versus-window interactions, and it recurs when the Qwen turbo endpoints hit the same wall in July (§13.3).

8.3 Data-access failures are filtering reasons, not results

Several OLMo benchmarks could not be run at all, and classifying them correctly was the point:

- `competition_math` was disabled on the HF Hub (script-based dataset, 401 on the loader).
- `natural_qa`: the public GCS bucket revoked *authenticated* reads (it dropped `allAuthenticatedUsers`, not just `allUsers`), so no credential unblocks it. An entire staging shim built to work around the assumed "authenticated reads work" had to be reverted once the access assumption was falsified by a single test with real credentials. The recorded lesson — *validate the access assumption before building the plumbing that depends on it* — is a methodological point for the paper's failure taxonomy.
- `gpqa` is a gated HF dataset (requires accepting terms).

Each was disabled as a documented *environment* failure, joining a taxonomy that is deliberately kept separate from the reproducibility verdicts.

8.4 The canonical-key matching bug: 114 phantom misses

The most consequential "bug" of the OLMo campaign was not in execution but in *pairing*. The planner matched logical-key *strings* by generating separator permutations and (under a flag) stripping `groups=` tokens, but it never canonicalized *token order*. The OLMo MMLU keys are the same token set in a different order, so the local and official variant sets had an *empty intersection*: all 114 MMLU packets were reported as `missing_official_component` even though the public counterparts existed in the index.

The fix (`docs/planning/core-report-planner-robust-matching-plan.md`) replaced the order-sensitive matcher with a single `canonical_logical_key` in `eval_audit/run_entries.py`: parse the `benchmark:k=v,...` name, drop bookkeeping-only tokens (`groups`, `model_deployment`), canonicalize values (model `/`↔`_`, `mmlu_pro` `subject` → `subset`), and re-serialize with the key–value pairs *sorted by key*. Once you sort, the separator-permutation and groups-strip variants all fold into one deterministic string, so keeping both matchers would have re-created the very "two competing

normalizers" divergence the fix set out to end. The real-data validation expectation: `n_skipped` 114 $\to\sim$ 0, `n_built` 35 $\to\sim$ 149.

This is a symmetric equivalence, correct for *grouping*; the asymmetric subset test (`run_dir_matches_requested`) was deliberately preserved for `compare_batch`, which answers the different question "does this candidate satisfy this request." The distinction — symmetric equivalence for grouping vs asymmetric subset for satisfaction — is a reusable design point.

8.5 The BBQ prompt drift: a genuine recipe disagreement

Not every difference is an artifact to be fixed. The BBQ runs surfaced a *real* recipe drift: the official run entry carried `output_format_instructions=mcqa` and the prompt began "Answer with only a single letter." (followed by the standard preamble) while the locally reconstructed run entry dropped that token and produced only "The following are multiple choice questions (with answers)." The model sees a different prompt and produces measurably different output. This is gotcha G5 (§A), and it is precisely the kind of difference that must be *surfaced* (via the `same_instructions` comparability fact), not silently fixed — and it is one of the strongest motivations for the from-spec replay of Chapter 10: replaying the official `run_spec.json` verbatim makes such reconstruction drift vanish by construction.

Chapter 9

Containerization and GPU Leasing

Commits: 2026-06-16 through 2026-06-24. Two intertwined infrastructure efforts: pinning the execution environment in Docker, and letting many HELM runs fan out across GPUs under a lease.

9.1 Why containerize: the environment is a variable

By default `eval-audit-run` executes each HELM run-entry in the host venv of whatever GPU machine the scheduler lands on, making torch / CUDA / transformers / HELM versions an *uncontrolled variable* — exactly the kind of drift the audit exists to separate from residual disagreements under controlled equivalence. The containerized path runs each run-entry inside a pinned Docker image and records *which image, by `sha256` digest, produced each run.*

The design (confirmed interactively) is worth recording:

- A brand-new, independent multi-stage Dockerfile (CUDA devel→ runtime, `uv`-managed standalone CPython, venv at `/opt/venv`), built from *pristine committed submodule state* via `git archive` at each submodule's gitlink SHA — no remote dependency, no `.git` leakage, SHAs recorded as OCI labels.
- The image tag is resolved to an immutable `<repo>@sha256:<digest>` reference *once at schedule time* and every node is pinned to it, so a rebuild automatically re-pins.
- The container runs as root; a thin entrypoint chowns *only* the output dir back to `HOST_UID:HOST_GID` on exit, so `kwdagger` on the host can own results while in-container HF-cache writes still succeed.
- The docker-wrapping `kwdagger` node *subclasses* the magnet materialize node and overrides only `command`, inheriting the `out_paths`/`DONE`-sentinel contract for free — the reusable lesson being "wrap an existing scheduler node in a new execution substrate by subclass + override, not reimplementation."

Several first-real-build bugs are instructive because each could only surface at the first genuine build or run and each maps to a general lesson:

- **Dangling interpreter symlink** — `uv`'s managed standalone Python was not copied into the final stage, so `/opt/venv/bin/python` was a dangling symlink reporting "command not found." Lesson: *uv-managed Python + multi-stage Docker = copy the interpreter, not just the venv*, and put the import smoke-test in the stage that ships.
- **HF token not reaching the container** — `kwdagger` runs each job in a fresh tmux pane whose environment is deliberately empty (secrets hygiene), so `-e HF_TOKEN` forwarded nothing; the host-venv path had silently relied on the on-disk `huggingface-cli login` token. Fix: write the resolved token to `<hf_cache_dir>/token` (mode 0600) at schedule time, in the

process that *did* inherit the env. Lesson: *a redundant channel that silently carries the load hides the breakage of the primary one.*

- **Wrong extras and an unpinned hub** — the image installed `crfm-helm[heim]` (image-generation extra) which lacks `langdetect` (needed by ifeval), and left `huggingface_hub` unpinned so it floated to a version whose repo-resolution API broke `wmt_14`. Fix: install `crfm-helm[all]` and co-pin `huggingface_hub==0.36.2`, asserted in *both* the install layer and the final stage. Lesson: *pin the secrets-of-resolution (transitive floats), not just the top-level package*, and pair a build-time guard with a runtime probe of the resolved artifact (a stale pinned digest is invisible to the build guard).

9.2 From profiles to a leasing catalog

Mid-June the `infer_stack` submodule was rewritten from a profile/contract model to a *catalog* (models + endpoints) plus *leasing* (`acquire`/`release`) model. The two in-scope runbooks (phi-2 e2e and the six OLMo presets) were migrated: `load_profile_contract` was replaced with a pure static `resolve_serving_facts` over the catalog; the config dirs were reschema'd from `config.yaml`+`models.yaml` to `settings.yaml`+`catalog.yaml`; and an explicit `protocol_mode` (completions for base models, chat for instruct) was made a required preset fact rather than a silent default. A subsequent question — "do `serve` and `acquire` both need to exist?" — collapsed the two verbs into `acquire` once a one-line grep proved the distinguishing `owner='manual'` field drove no behavior.

9.3 The leasing pipeline: preconditions as job phases

The deeper design question was how to drive *high-parallelism* reproduction: each HELM run is a `kwdagger ProcessNode` that must bracket itself with a GPU lease (acquire before, release after), where release must run after finish *or* crash. The decisive argument, reached across three rounds of design discussion, was *co-location*: the lease lives in a node-local ledger and the gateway is localhost, so release must run *where* acquire ran. A separate cleanup DAG node cannot promise that (and is actively wrong on multi-node scheduling), and success-only dependencies (`afterok`) skip a release node when the run fails — the exact leak to avoid. The right primitive is a job *phase* (setup/teardown), which is also the more general, reusable scheduler feature.

The findings that settled the design were verified against source, not assumed: `cmd_queue`'s `preamble` already gates the main command, so only an always-run `teardown` trap was new; `kwdagger`'s cache guard (`test -e DONE || cmd`) means a *skipped* job acquires nothing; and a blocking acquire's retry loop *reclaims TTL-expired leaks while it waits*, so queue-and-wait and leak-recovery become one mechanism — *iff every pipeline lease has a finite TTL*. A subtle bash-precedence trap was caught and closed: a setup chain `acquire && { snapshot } || true` lets a *failed* acquire fall through to `|| true`, defeating the gate; the fix scopes `|| true` inside the snapshot brace so only the snapshot is best-effort.

Two further design moves are worth recording because they generalize:

- **Decoupling leasing from containerization.** The lease bracket was initially bolted onto the docker node only, which would have dragged containerization onto the host-venv vLLM scenarios. It was extracted into a transport-agnostic `LeaseBracketMixin` + shared command renderer; leasing and containerization became *orthogonal* axes (a run can be docker+lease, docker+no-lease, or — briefly — bare+lease). The lesson: when a capability seems "coupled" to

a transport, check whether it is the capability that needs the transport or just where someone bolted it on.

- **Scoped cleanup over global teardown.** Every `release -all -evict` was removed from the runnable path in favor of per-run release (`reclaim: stop`) plus a scoped `infer-stack gc` that reclaims only TTL-expired leaks in this user's ledger — so the shared docker stack is never torn out from under a co-tenant. A bootstrap that exists only to read a managed secret pays for it with a scoped `-env-file` handle, not a global teardown.

By the end of this arc, containerization was made *mandatory* and the non-containerized code paths were deleted, with leasing the orthogonal opt-in — so the execution environment is always a pinned, digest-recorded artifact (`container_provenance.json`).

Chapter 10

Faithful Replay: From Run-Entry to Run-Spec

Commits: 2026-06-25 through 2026-06-30; the ***impl/run-from-run-spec*** *branch on which most subsequent work sits.*

10.1 The core methodological principle

This chapter contains what is arguably the single most important methodological decision of the internship, and it should anchor the paper's "methods" section.

Public HELM runs do not record the CLI run-entry string that produced them, nor the exact HELM version. What they *do* ship is a fully resolved `run_spec.json`: the complete adapter spec, scenario spec, metric specs, and annotators, with every default already materialized. The prior approach — and the approach every earlier reproduction took — was to *reconstruct* a run-entry string from the run and re-parse it. But re-parsing re-derives the recipe under *today's* library defaults, which silently drifts from the recipe that actually produced the official numbers (this is the mechanism behind gotchas G1, G5, and the BBQ drift of §8.4).

The principle: never reconstruct the recipe. Replay the resolved ***run_spec.json*** verbatim.

Concretely, a new `materialize_helm_run_from_spec.py` CLI (a faithful- replay sibling of the run-entry materializer) reuses the discovery/matching scaffolding but swaps the compute step from "reconstruct a run-entry string → `helm-run` subprocess" to "`from_json(run_spec.json)` → in-process `run_benchmarking`." In discovery mode, the run-entry string now does nothing but *locate* the official directory; the authoritative resolved recipe drives execution. This memory — *all reproductions must be from-spec* — became a standing rule, and the older OLMo run-entry path was flagged for migration.

10.2 Two failure modes, one loud and one silent

The design carefully distinguishes the two ways a from-spec replay can go wrong, because they demand opposite handling (this is gotcha-adjacent and reviewer- relevant):

Class drift is loud.
If the `run_spec.json` references a scenario or metric class that does not resolve under the local

HELM (`get_class_by_name` raises `ImportError`), the preflight catches it before any instance runs. This is the correct behavior: a class that cannot be resolved is an environment mismatch, and it is surfaced as such. (The preflight's exception net had to be broadened from `(ImportError, AttributeError)` to `Exception`, because importing a vision–language scenario raises HELM's `OptionalDependencyNotInstalled` — itself a recipe/environment filtering reason, not a crash to propagate.)

Field drift is silent.
The cattrs codec ignores unknown keys and fills defaults, so a dropped field produces no error — only wrong numbers. The guard is a round-trip test: because HELM *writes* `run_spec.json` with `asdict_without_nones` (dropping only `None` values), every on-disk key is non-`None`, and the re-serializer re-emits every non-`None` field — so a key missing after `from_json`→`to_json` is an unambiguous silent drop. Ten sampled MMLU specs round-tripped with zero key loss.

The reusable insight recorded at the time: *writer/serializer asymmetry decides whether a diff test is sound* — the no-key-dropped assertion holds only because the writer drops a superset of what the re-serializer drops.

10.3 Making the deployment substitution visible

A from-spec replay serves the model on local hardware, which is by definition a *different deployment* than the original (e.g. a hosted Together API). The subtle danger is that a pure by-name replay would record the *official* deployment name in the produced run, so the comparison would report `same_deployment=yes` and the single most important difference the audit exists to surface — that the serving engine changed — would become invisible.

The fix (`from-spec-deployment-rewrite-plan.md`) rewrites `adapter_spec.model_deployment` to the *local* name after deserialization, via an explicit `-model-deployment` argument. Explicit, not auto-derived: the whole feature exists to stop *silently* masking the substitution, so its own failure mode must be loud — a wrong name is a HELM "deployment not found" crash *before any instances run*, whereas auto-derive's failure mode is the silent no-op that fails *toward* the very bug it fixes. The produced run then records the served endpoint, and the existing diff logic correctly reports `same_deployment=no` with no downstream plumbing.

10.4 Addressing runs by (root, relative-path) and content-addressed caching

A further refinement replaced the drift-prone run *name* as the addressing mechanism with *host-side* resolution by `(precomputed_root, rel_path)`: resolve the official `run_spec.json` at schedule time, apply *raw-JSON* scalar edits (only `adapter_spec.model_deployment` and `max_eval_instances`, so no cattrs round-trip and therefore no field drift), and materialize a substituted copy under a staging directory. Each run rides one `kwdagger` *submatrix* entry (the verified zip primitive) so the per-run tuple travels together without an $N \times N$ cross-product. Two consequences worth recording:

- **No image rebuild, no corpus mount.** The runner needs only the tiny staging directory, because substitution happens host-side.
- **Content-addressed caching.** `max_eval_instances` is baked into the materialized copy but is not a matrix parameter, so a naive fixed filename (`run_spec.json`) would make `kwdagger` *skip* and reuse a stale run after a cap change. The fix appends a content hash to the

filename: `run_spec.<sha256(bytes)[:16]>.json`. Because the materialized path *is* the node's algorithm identity, an identical recipe yields the same path (skip) and any changed field yields a new path (recompute). This is the reusable "content-address generated input files so baked-in changes invalidate the cache" rule.

An exporter "auto-freeze" resolves each preset run-entry to its exact rel-path *once* at export time (against the corpus snapshot) and freezes the sources into the generated manifests — the only remaining place token-subset discovery runs, pinned where drift cannot occur — and a dry-check existence mode validates that a frozen manifest's rel-paths still resolve.

Chapter 11

Deployment Matching: The Flagship Investigation

Commits: 2026-06-30 through 2026-07-10; the `dev/tools/deployment_match/` tool and the documents `vllm-vs-huggingface-deployment-match.md` and `helm-unrecorded-deployment-params.md`. This is the scientific heart of the internship: the sustained effort to attribute residual disagreement to a specific, unrecorded execution parameter.

11.1 The provocation: OLMo-7B produces prompt-independent gibberish

The investigation began with a dramatic symptom. During from-spec reproduction of `narrative_qa`, local OLMo-7B repeated the same prompt-independent boilerplate — "The first thing you need to do is to make sure that you have a... before you start playing." — for *every* instance, while the official `together/olmo-7b` conditioned on the prompt correctly. The diff showed `prompts_equal=True` (the prompt reached the model) and a hugely negative local completion log-prob against the official's 0: the model ran but ignored its input.

The first hypothesis was a documented one: OLMo-1 7B `float16` numerical instability (activation outliers exceed fp16's range → inf/NaN in attention → collapse to the highest-prior pretraining attractor). A minimal deployment matrix was built to test it — and the hypothesis was *refuted*. The true cause was a *tokenizer* issue: `OLMo-7B-hf`'s tokenizer appends an `<|endoftext|>` token to the prompt under vLLM's default `add_special_tokens=True`; the Together deployment did not do this, and HELM never recorded the difference. The production fix serves the OLMo-1.7 tokenizer (whose post-processor omits the append), and the completions path forwards `add_special_tokens=false`. **Finding.** This is the paradigm case of the whole project.

A disagreement that looked like flat irreproducibility (the model outputs nonsense) was in fact a *recipe/environment* failure caused by a single unrecorded request-time tokenizer flag. And the first, plausible theory (fp16 instability) was wrong — which is exactly why the response was to build a *sweep* rather than trust a single-knob theory. The lesson — that single-knob theories about deployment divergence had by now been wrong twice — became the design charter for the deployment-match tool.

11.2 The tool: sweep, don't guess

`dev/tools/deployment_match/` takes *any* public HELM run, builds a grid of plausible local deployments for that run's model, evaluates a small length-stratified prompt sample, and ranks each cell by closeness to the official completions. Because Together stores no log-probs, the objective is *agreement with the official completion text* (a composite of quasi-exact- match, first-word match, and similarity, with a model-agnostic "prompt-independence collapse" detector). The grid is two-tier: serve-time knobs (dtype × tokenizer × `max_model_len`) become distinct infer-stack endpoints; request-time knobs (`add_special_tokens`, `add_generation_prompt`, protocol) are probed per request against one container. The tool has `dry-run` / `run` / `confirm` / `auto` phases, and `confirm` produces a real one-endpoint catalog you can `acquire`.

A key design principle, formalized after the fp16 mis-diagnosis, splits "exclude settings beforehand" into two opposite treatments:

- **Feasibility pruning (safe).** A recipe that physically cannot serve on this hardware teaches nothing and wastes a full serve cycle, so it is dropped a priori with a typed reason (mirroring the Stage-1 filter pattern). Example: `float32` on a MoE model → `infeasible:moe-fp32-shared-mem` (see §11.4).
- **Relevance pruning (risky) is refused.** "This knob probably won't change the output" is exactly what the tool will not guess; such knobs stay in the sweep unless the operator narrows an axis explicitly.

11.3 Chat-template version drift: `add_generation_prompt`

The next finding is a subtle, real reproducibility trap on OLMoE-instruct, and it is a clean example of a *software-version* dependency hiding inside a recipe. HELM's `get_prompt` always calls `apply_chat_template(..., add_generation_prompt=True)` — but that flag only does anything if the chat template *implements* it (`{% if add_generation_prompt %}<|assistant|>\n{% endif %}`). The OLMoE template shipped with HELM's *older* transformers *ignored* the flag, so `add_generation_prompt` was effectively *False* and the model was fed the prompt *without* the trailing `<|assistant|>\n`. Modern transformers/vLLM ship an updated template that honors it, so the same call now *appends* the suffix → a different prompt → a different output on every cell. Rendering with `add_generation_prompt=false` on modern transformers moves the output back toward the official, confirmed empirically. The tool now sweeps `add_generation_prompt` ∈ {True, False} as a cheap request-time knob and lets the scorer decide.

11.4 The float32 discovery: a flagship attribution pilot

> **Authoritative status.** This section reports two distinct forms of evidence. The source/configuration path makes fp32 the most likely historical dtype under bounded pre-v5 assumptions. Separately, a 12-instance IFEval probe found float32 candidate cells with the highest sampled agreement. Neither establishes the complete historical deployment, and the candidate has not yet been confirmed through the ordinary HELM path on held-out data.

The single most consequential finding of the internship concerns model *precision*, and it is the one most worth foregrounding in the paper.

HELM's `HuggingFaceClient` loads a model with only the kwargs the deployment config supplies. For `huggingface/olmoe-1b-7b-0125-instruct`, the config gives exactly `args: {device_map:`

`auto}` — *no* `torch_dtype`. So `AutoModelForCausalLM.from_pretrained(..., device_map="auto")` is called with no dtype, and *every transformers 4.x* defaults that to `torch.float32` for backward compatibility, *ignoring the checkpoint's bf16 config.* (Reading the checkpoint's dtype is a transformers v5 change; even late 4.x still hits "`else: set fp32 as the default dtype for BC`.") The OLMoE architecture landed in transformers ∼4.45, so a January-2025 run is squarely in the fp32-default regime. This is a *deductive* claim — read off the loader source and the library's version-dependent default, not fit to outputs — so it is stronger than an observational match. Its residual is the one parameter it depends on: the exact `transformers` version, which the run dir does not record (only bounds). Under that reading:

> *The code path* HELM *used loads the weights in* `float32` *under every* `transformers` *version consistent with the OLMoE run's date — so the official most likely ran* `float32`*. (Conditional on* `transformers`*<5; a run produced under ≥5 would default to bf16 and move this conclusion.)*

A small oracle-scored *probe* corroborates this: reproducing the official on the HF side at `float32` (`transformers.generate()`) reaches quasi/exact match ≈ 1.0 on the sampled instances, versus ≈ 0.17 for the best fp16 vLLM cell. Table 11.1 shows the overnight probe sweeps across the four OLMo instruct models. **This is deployment-*matching* evidence, not a completed HELM reproduction** (see the epistemic-status note below): the scores are composites over $n \approx 12$ ifeval instances, scored against the sampled official completions, and the winning cells additionally rely on a probe-only `add_special_tokens=False` request knob that an ordinary HELM run does not send.

Model	HF best	vLLM best
OLMoE-1B-7B (MoE)	**fp32 MATCH 0.971** (quasi 1.0)	fp16 PARTIAL 0.494
OLMo-2-7B (dense)	fp32 PARTIAL 0.175 (quasi 0.0)	**fp32 MATCH 0.915**
OLMo-2-13B (dense)	fp32 PARTIAL 0.324 (quasi 0.0)	**fp32 MATCH 0.904**
OLMo-2-32B (dense)	fp16 PARTIAL 0.234 (quasi 0.0)	**fp32-tp2 MATCH 0.961**

Table 11.1: Overnight deployment-match sweeps ($n = 12$ ifeval instances, scored against the sampled public completions). Float32 is the highest-agreement tested precision; the winning *engine* depends on architecture. OLMoE is MoE, so vLLM's Triton fused-MoE kernel cannot serve fp32 (shared-memory limit) and only HF fp32 produced the exact sampled OLMoE match; the dense OLMo-2 samples matched most closely under vLLM fp32. These are probe results pending ordinary-path confirmation.

Epistemic status of the fp32 result (interpretive revision, 2026-07-15). This table is a *configuration-discovery* result, not a reproduction result. Read it as: *among the deployment configurations tested,* `float32` *candidates produced the highest agreement with the sampled public OLMo outputs.* It does *not* yet establish "the historical deployment was `float32`" as an identified fact, nor "`float32` reproduced the official HELM result," for three reasons: (i) matching a small oracle sample does not uniquely identify the producing deployment — multiple latent configurations can be observationally equivalent on $n \approx 12$; (ii) the winning cells use a probe-only `add_special_tokens=False` knob not sent by an ordinary HELM run; (iii) the "confirm" step — land the winning config on the real HELM execution path, run the exact public `run_spec.json`, and compare through the standard pair-report pipeline on *held-out* instances — has not been run for any model. The *deductive* dtype-default argument above is the stronger leg and survives these caveats (it reads the loader, not the outputs), but it too is conditional on the unrecorded `transformers` version. See Appendix D for the consolidated epistemic status and the corrected disagreement taxonomy.

Two aspects make this finding sharp:

- **The matching precision is the one cell that cannot run.** vLLM's Triton fused-MoE kernel needs ∼131 KiB of shared memory per block in fp32 — over the ∼99 KiB cap on workstation cards — so it OOMs at the *kernel* level (not VRAM). Every runnable local vLLM cell for OLMoE is reduced-precision, and none matches the sampled high-precision reference as closely. That *fp16 beats bf16* among those cells is itself corroboration: fp16's finer mantissa flips fewer greedy near-ties against a high-precision reference, which says the reference is higher-precision than bf16 — consistent with fp32. Serving fp32 requires either an H100-class card (big shared memory) or, for dense models, tensor-parallel fp32 (`-fp32-tensor-parallel-size 2`); for the MoE, HF `generate()` at fp32 is the only route.
- **It reframes the disagreement.** A bf16-vs-official OLMo-2 disagreement is a *deployment-boundary artifact* (a precision the local engine could not easily provide), *not* a reproducibility failure — fp32 is the leading tested candidate for closing it, pending ordinary-path confirmation. This is precisely the recipe/environment-vs-reproducibility distinction of §3.2, applied to the most consequential single case.

11.5 The residual puzzle: unresolved Hugging Face divergence for dense OLMo-2

Rigor demanded following up an inconsistency. The HF probe exactly matched the sampled OLMoE outputs but gave near-zero quasi-match for the three dense OLMo-2 models, even though vLLM fp32 best matched the sampled dense-model outputs. The forensic diagnosis localized the disagreement: tokenizing the exact ifeval prompt showed byte-identical tokens across every path (so it is *not* the prompt); yet the HF probe's first token disagreed with the official (first-token agreement 0.42) while vLLM fp32 matched the sampled official text character-for- character (first-token 1.00). A first-token disagreement on a byte-identical prompt is a clean litmus: at step 0 there is no accumulated history, so a different first token can only mean a different *forward-pass* logit vector. The divergence is a fp32 forward-pass difference, not a prompt or precision-class difference — attributable to knobs HELM never records: `attn_implementation`, device placement (`device_map=auto` shards a fits-on-one-GPU model across GPUs, changing fp32 reduction order), and the decode path (HELM's `do_sample=True, temperature=1e-7` vs the probe's greedy). The probe was extended to sweep these forward-pass axes. **Interpretive revision: this is a finding, not merely a broken probe.** An earlier framing dismissed the OLMo-2 HF divergence as "not a science problem" (a defect in our tool, since vLLM fp32 did match). The sharper reading, which the paper should adopt: a *current* `transformers.generate()` at fp32 does *not* reproduce a historical result *recorded as a `HuggingFaceClient` (in-process `transformers`) deployment — even under byte-identical prompt tokens*. That is precisely an *unresolved systematic divergence under prompt-token equivalence* — direct evidence of execution-stack sensitivity, and a core datum for the thesis, not an artifact to wave away. Its cause is not yet isolated, and two classes of explanation remain live until one is ruled out: a genuine execution-instrument divergence (candidates: model/tokenizer revision, `transformers`/`torch` version, attention implementation, generation defaults, device placement, numerical precision, CUDA behavior) *and* a defect in the dense-model HF probe path itself (e.g. tokenizer/template handling that differs from the OLMoE path on which the probe succeeds). The reframing rejects only the premature dismissal of the divergence, not the possibility that our own tooling contributes to it. That vLLM fp32 happens to match is a separate fact and does not make the HF→HF gap uninteresting. The reusable insight stands and is sharper for it: *"same engine family" (transformers→transformers) does not imply reproducible* — device sharding, attention-implementation drift, and sample-vs-greedy each perturb fp32 logits enough to diverge greedy

decoding.

11.6 Naming the gap: the unrecorded execution substrate

The investigation crystallized into a taxonomy (`helm-unrecorded-deployment- params.md`, Appendix B): HELM serializes the *recipe* and the model's *name* but almost nothing about the *execution substrate*. Empirically, of 148 HuggingFaceClient deployments in HELM's `model_deployments.yaml`, only 19 pin `torch_dtype` (all `bfloat16`) and only 7 pin a model `revision`. Ranked by effect on a greedy output, the unrecorded parameters are: (Tier 1) weight revision, load precision, quantization, software-stack versions; (Tier 2) attention implementation, matmul/TF32 precision, hardware, device topology, batch composition; (Tier 3) tokenizer + chat-template version, `add_special_tokens`/BOS handling, `generation_config.json` defaults, stop-sequence/detokenization semantics. The one high-leverage lever is *pinning the model revision (commit SHA)*, because the tokenizer, chat template, and generation config all live inside the model repo — so `model_revision` + `torch_dtype` + `transformers_version` together close the majority of the gap. Crucially, these must live in `client_spec.args` / `catalog.yaml` / a `recipe_facts` block, *not* in the run-spec name or `model_deployment` identity string — a SHA in the pairing key would break official↔local pairing, whereas a SHA in the provenance block is invisible to it.

A companion effort (`hf_inprocess.py` + infer-stack `acquire -reserve-gpus N`) built the mechanism to reproduce a HuggingFaceClient official *the same way it was produced* — in-process `transformers.generate()` at pinned fp32, on a GPU held (but not served) by a reserve-only lease. The mechanism landed; the routing switch that would make the default replay path choose it for a HuggingFaceClient official was scoped but left unwired (a replay still defaults to vLLM).

Chapter 12

Era-Pinned Containers: Reproducing the Pre-v0.5 Corpus

Commits: 2026-07-08 through 2026-07-13; the `impl/era-pinned-helm-containers` branch, the `docker/era_shim/` package, and gotcha G13.

12.1 The problem: the measurement instrument itself has versions

An internal planning estimate placed roughly **59%** of the then-indexed audit corpus on the classic-track `v0.2.4`/`v0.3.0` path (`docs/container-execution.md`; `dev/journals/claude.md`). The percentage must be regenerated from a checked-in corpus manifest with an explicit numerator and denominator before publication. Those runs *cannot* be replayed by the modern runner, which pins HELM 0.5.x and Python 3.11 and whose from-spec CLI imports v0.5+ module paths that do not exist pre-v0.5. This is not a model problem; it is a *harness* problem. And it matters for reproducibility science because the harness is the measurement instrument: the technical report had already shown that the exact `pandas` version (2.0.x vs 2.2+) flips per-instance row ordering on `entity_matching`, i.e. the harness's own dependency versions change the recorded instances.

The solution is to freeze the instrument at the era that produced each run: give each era its own *CPU-only* Docker image whose HELM harness is checked out at that era's release commit (v0.2.4 = `626d8609`, v0.3.0 = `8ea285f7`), with era Python (3.10) and era dependency pins, while keeping model inference *out-of-process* on a modern vLLM lease. The container thus holds only the scoring instrument fixed; the model server is shared with the modern path.

12.2 The design invariants

- **One manifest = one era = one image = one measurement instrument.** A mixed-era source set is a hard error at make-manifest time (`eval_audit/eras.py`). Era resolution keys on the path-derived `(public_track, suite_version)` signal.
- **Verbatim by-name replay.** A pre-v0.5 `adapter_spec` has no `model_deployment` field, so nothing is rewritten; routing to vLLM is purely by-name (the era shim registers a deployment under the exact official model name), and the materializer *refuses* to insert a `model_deployment` field into an era spec. Consequently `same_deployment` resolves `unknown` for era pairs (both sides lack the field) — the correct behavior, not a bug.
- **A standalone shim.** `docker/era_shim/helm_era_shim/` never imports magnet or eval_audit; it provides a flag-compatible from-spec CLI (`replay.py`) plus a backported OpenAI-compatible

completions client, so the era image depends only on era HELM.

- **An era↔image guard.** The image carries an `org.aiq.era` OCI label, checked against the manifest era at schedule time; a wrong pin fails loud.

12.3 The v0.2.4-versus-v0.3.0 API skew

A multi-angle review (verifying every era-API claim against the era HELM source via `git show <commit>:<path>`) found ten issues, the load-bearing ones being cases where the shim had been written against v0.3.0 semantics and broke on the older v0.2.4 API. Two are especially instructive:

- **Silent Together fallthrough (v0.2.4).** v0.2.4 lacks the register-deployments-from-path helper, so without an explicit `register_model_deployments_from_path` the deployments route to `TogetherClient` — silently reintroducing the very hosted-API artifact the audit exists to eliminate.
- **pyhocon dot-splitting (both eras).** pyhocon path-splits credential keys on `.`, so a dotted model name like `pythia-6.9b` dies at the credential lookup. The fix writes a nested-key `credentials.conf` and puts `api_key` in the client-spec args.

12.4 A turnkey validation runbook and the 8 GB subject

The era gates were rebuilt as `dev/era-tests/`, mirroring the `dev/e2e- tests/` conventions, so one model runs end-to-end through both classic eras with near-zero setup.

> **Packaged-evidence status.** By July 14 the journal refers to intact `era-redpajama` runs and refreshable report artifacts for both proxy eras, and the final slide says the end-to-end script works. Those report stores live under `/data`, which the supplied source archive excludes. This master therefore records the execution as locally supported but requires the report directories, inputs, and hashes to be archived before publication.

The subject was swapped from `eleutherai/pythia-6.9b` (∼14 GB, does not fit 8 GB) to `together/redpajama-inci` the smallest corpus model with a *full 74-run packet at both eras* (∼2.8B params, ∼5.6 GB fp16). A "validation ladder" checks the instrument in rungs: image sanity; instrument fidelity on a pandas-sensitive `entity_matching` run (byte-for-byte instance identity, no model); an end-to-end single run; a full packet per era; and a per-scenario HF-fetch audit. Several first-real-run bugs (a build-time import that named the wrong module for the era; a `pandas`/`pyarrow` dependency-resolution skew; a fidelity rung that diffed a `scenario_state.json` the public corpus never ships — switched to the always-present `display_requests.json`) reinforced the theme that some bugs only surface at the first genuine build.

An important taxonomy point resurfaced here: rung 2 originally *failed* on scenarios whose data is no longer fetchable (e.g. MATH's script loader returns 401 on the 2026 Hub). That is an *environment* filter, not an instrument failure, so the rung was changed to classify data-fetch/auth crashes as SKIP — the same recipe-vs-reproducibility distinction, now applied to a historical-data gate.

12.5 G13: a class path that resolves in no released HELM

The deepest archaeological finding of the internship concerns the original-paper classic models (GPT-J 6B, GPT-NeoX 20B, OPT-66B) and is written up as gotcha G13 (§A). Era-replaying these fails the shim's class preflight: their `run_spec.json` names `helm.benchmark.basic_metrics.BasicMetric`,

which resolves in *no* released HELM version. An investigation over a blob-less clone of the HELM repository established that the stored path is a once-migrated hybrid: a bulk `benchmark.`→`helm.benchmark.` prefix rewrite (at the `src/helm/` rename, 2022-11-16) applied to a run-spec whose metric class was still *flat* at production time. Reversing the naive migration triangulates the producing code to a ∼4-week window (2022-07-31 to 2022-08-26) of *unreleased pre-v0.1.0 HELM* — after scenarios were nested but before metrics were. The class-path *shape* (flat vs subpackage) is thus a sharper origin fingerprint than release dates. Because no released version resolves these, and building the true origin instrument is infeasible, the fix is a self-verifying declared class-path canonicalization in the era shim: when a flat `helm.benchmark.*` class fails to resolve *and* its `helm.benchmark.metrics.*` relocation resolves to the same leaf, remap it on the in-memory run-spec (recorded as a declared substitution). It fires only for the known relocation — a genuinely wrong era pin still fails loudly.

A related provenance fact was pinned along the way (memory *classic-officials-carried-forward*): the gptj/neox/opt/redpajama official runs are *byte-identical* across v0.2.4 and v0.3.0 — carried forward, not re-run — so "two eras agree" is one public number measured by two instruments, not two independent confirmations.

Chapter 13

New Model Families: GPT-OSS and Qwen

Commits: 2026-07-11 through 2026-07-14; from-spec runbooks for `openai/gpt-oss-20b` *and a combined Qwen fan-out.*

With the from-spec, containerized, leased pipeline mature, two further model families were brought in as reproduction subjects — and as a test of whether the machinery generalizes beyond OLMo.

13.1 GPT-OSS-20B and the null-content crash

The public GPT-OSS-20B rows were served via `together/gpt-oss-20b`, a chat (harmony) client, so the faithful replay serves *chat*, not the frozen older preset's completions workaround. The design tension: GPT-OSS is a reasoning model, and a chat turn can return `message.content=null` (reasoning-only, `finish_reason=length`), which un-patched HELM crashes on. The root cause was pinned precisely (memory *gpt-oss-null-content-root-cause*): the crash is a *null-versus-empty-string serving difference* — Together returns `""`, vLLM returns `null` — and normalizing `null`→`""` is faithful (`max_tokens` and window were ruled out as causes). The fix makes the null-safe chat clients normalize `content:null`→`""`, leaving HELM untouched. Discovery confirmed all four target run directories resolve 1:1 against the 84,966-run corpus with 0 ambiguities.

> **Store status (collaborator-verified, not packaged).** A live-store review reports that the `gpt-oss-20b-from-spec` *reports* were regenerated on Jul 14 at 13:47 after the drift-plot fixes. This supports report freshness, but a report timestamp alone does not establish that the underlying model outputs and comparison inputs were generated by the final accepted execution path. Before these numbers are cited, preserve and hash the store, frozen run specs, public targets, model/deployment manifests, code commit, and comparison configuration, then run the acceptance checks in the canonical store-status ledger.

Finding. GPT-OSS-20B is a promising positive local case.

The headline aggregate score-drift plot (Figure 13.1) shows local and public headline scores agreeing closely across all four benchmarks — `bbq` 0.967/0.963, `gpqa` 0.594/0.610, `ifeval` 0.732/0.754, `mmlu_pro` 0.740/0.748 (public/local) — with squared errors bounded by $\sim 4 \times 10^{-4}$, more than an order of magnitude ($\approx$30$\times$) tighter than the OLMo instruct drift on `ifeval` ($\sim$1–1.6 $\times 10^{-2}$). This

is promising evidence that family-specific deployment and artifact semantics matter. It becomes a publication result only after the underlying run and comparison provenance are preserved, hashed, and audited; report regeneration alone is not a sufficient acceptance criterion.

[Figure: "Headline Aggregate Score Drift" heatmap for `experiment_name=gpt-oss-20b-from-spec`. One model row (`openai/gpt-oss-20b`) × four benchmark columns (bbq, gpqa, ifeval, mmlu_pro), each cell annotated `P <public> / L <local>`, colored by $(\text{local} - \text{public})^2$. Source: 07-14 status deck.]

Figure 13.1: The locally reported GPT-OSS-20B store closely matches the sampled public headline metrics (squared error $\leq 4 \times 10^{-4}$). The rendered artifact is `aggregate_score_diff_headline.png` produced by `build_reports_summary`.

13.2 Qwen: generalizing the combined fan-out

> **Workflow status.** The combined Qwen presets, serving fan-out, discovery, and report plumbing were implemented. The packaged source archive does not contain a fresh, fully validated final exact-path report store for the complete eight-model roster. Treat model-level performance claims as pending until the store is regenerated and its raw run directories are preserved.

A combined exact-spec workflow was prepared for the eight public HELM Qwen text models as a single `qwen-combined` multi-deployment from-spec fan-out — the `allenai-olmo-combined` analogue. The OLMo-specific "combined preset" helpers were generalized into `_combined_run_entries` and `_build_combined_preset` so both families share one code path (safe to generalize precisely because the OLMo combined tests pin the old output; regenerate and diff the entry count).

Three decisions carried over as reusable rules:

- **Membership rule (the olmo-7b lesson).** Model size is *never* a reason to exclude a member from the fan-out — large models serialize under the lease while small ones co-host; the *only* thing that forces a member split is discovery *ambiguity* under the shared `precomputed_root`. Propose all members and let the freeze preflight decide.
- **The whitelist is exactly reconstructible.** Filtering `official_public_index.csv` to classic-core + capabilities lands on the plan's per-model counts to the row ($85 \times 5 + 86 + 132 \times 2 = 775$), with two non-obvious exclusions (`banking77`, `bigcodebench`). All 775 rows resolve with 0 ambiguities, so no member splits.
- **Protocol is answerable, not "to confirm."** HELM's own `model_deployments.yaml` (read from the installed wheel) says base Qwen1.5 = `TogetherClient` (completions) and the chat/-turbo models = `TogetherChatClient` (chat) — so the serving protocol was resolved from the authoritative source, not guessed.

13.3 The Qwen turbo window bug

The last dated event of this chronology is a clean recurrence of the OLMo chat-template-window interaction (Chapter 8), now on the Qwen turbo endpoints. An `ifeval` smoke run on `qwen/qwen2.5-72b-instruct-turbo` died with a `ContextWindowExceededError`: the endpoint served `max_model_len: 4096`, but the two `*-instruct-turbo` endpoints are the only Qwen members that replay the capabilities whitelist (`ifeval`, `mmlu_pro`), whose official `run_spec.json` sets `max_tokens=4096`. HELM's window service truncates the *prompt* but never clamps `max_tokens`, so 4096 output tokens fill the entire 4096

window and vLLM 400s with no room for a single prompt token. The official Together serve ran a `max_sequence_length` of 128k, so the overflow never surfaced upstream. The scoped fix raises those two endpoints to `max_model_len: 8192` (4096 output + 4096 prompt headroom). This is a serving configuration bug, not a reproducibility failure — and it is a fitting last example of the internship's recurring theme: the recipe under-specifies the serving environment, and faithfully replaying the recipe surfaces exactly where that under-specification bites.

Chapter 14

Reporting and Analysis Infrastructure

Commits throughout, concentrated 2026-07-08 through 2026-07-14. The reporting layer is where reproducibility becomes legible, and much of the final fortnight was spent making the aggregate story honest and comparable.

14.1 The pipeline the reports run on

The analysis pipeline is read-only over audit results that already exist on disk ("no model is run; no benchmark is downloaded"). Its stages are: (1) convert both public and local runs to the EEE artifact format (`eval-audit-prepare-eee`); (2) compose a *virtual experiment* from a YAML manifest (`eval-audit-build-virtual-experiment`), applying a scope and computing the coverage funnel; (3) per-packet core analysis (`eval-audit-analyze-experiment`) through `NormalizedDiff`; (4) aggregate publication (`build_reports_summary`). A *virtual experiment* is the organizing abstraction — a declarative slice over existing audit data — and the *packet* is the planner's canonical comparison unit (a bundle of normalized components plus the comparisons to render between them).

14.1.1 The three-level coverage funnel

The coverage funnel is the quantitative backbone of any "fraction reproducible" claim and reports three nested levels (Table 14.1). The reason all three exist is gotcha G1: the raw `run_spec_hash` yields *zero* matches across HELM releases even for semantically identical recipes, because newer HELM populates `adapter_spec` fields older HELM left absent. The *canonical* recipe hash collapses those known schema-evolution fields before hashing; the gap between raw and canonical match is HELM-version churn, and the gap between canonical and logical match is real recipe drift.

Level	Meaning
logical	same scenario + model + augmentation (order-insensitive canonical key)
recipe-canonical	+ same scenario spec, prompt, decoding, `max_train_instances`, after schema-collapsing the run spec
recipe-identical	byte-for-byte `run_spec_hash` match

Table 14.1: The three-level coverage funnel. The report shows all three so that HELM-version churn (raw→canonical) is separated from genuine recipe drift (canonical→logical).

14.1.2 Sankey diagrams and the failure taxonomy

Two Sankey diagrams render the funnel: Sankey A (Universe→Scope) narrows the 13k+ discovered runs to the in-scope set through ordered gates (structural, metadata, open-weight, tag, deployment, size, manifest scope); Sankey B (Scope→Reproduced→Analyzed) carries in-scope rows through logical match, recipe-canonical match, analyzed packet, and finally an agreement bucket. Crucially, the failure-taxonomy colors and categories were made *honest*: environment/ recipe exclusions and controlled-equivalence outcomes are distinct categories, so a gated-model exclusion never reads as a reproducibility failure. This is the visual realization of §3.2.

14.2 The aggregate score-drift heatmaps

The final fortnight built the plots that summarize reproducibility at a glance — and, tellingly, most of the work was in making them *correct and comparable* rather than in drawing them.

- **Per-metric aggregate-score-difference heatmap.** Instead of instance-level agreement, this shows, per core metric, the difference between the reproduced (local) and official (public) *aggregate* scores; color is the signed difference on a symmetric diverging colormap (so 0 is the white "no drift" midpoint), each cell annotated `P <public> / L <local>`. A deliberate data-source decision: read the signed means from the already-written run-level CSV (`core_runlevel_table.csv`) rather than re-serializing every packet's JSON — it works on existing outputs with no re-render.
- **Headline holistic heatmap.** One plot showing every model×benchmark pair, each benchmark using its *headline* metric. `eval_audit` had no primary-metric concept, but HELM does (`run_groups[].environment.main_name` in the schema), so a curated `HEADLINE_METRIC_BY_BENCHMARK` map (mmlu→exact_match, narrativeqa→f1_score, wmt_14→bleu_4, ...) is used *iff* that metric is actually present, falling back to a priority list otherwise; the plot names the actually-used metric on each axis tick, so the choice is transparent. The final version colors by squared error $(\text{local} - \text{public})^2$ (non-negative, emphasizes big drifts) and orients models-left / benchmarks-bottom to match the coverage matrix. Figures 13.1 and 14.1 are instances.

[Figure: "Headline Aggregate Score Drift" heatmap for `experiment_name=olmo-models-combined`. Six model rows (olmo-1.7-7b, olmo-2-{7b,13b,32b}-instruct, olmo-7b, olmoe-1b-7b-instruct) × ∼12 benchmark columns. The base `olmo-7b` row reproduces the classic benchmarks near-exactly (GSM 0.036/0.036*, MMLU* 0.295/0.287*, narrative_qa* 0.597/0.595*, wmt_14* 0.097/0.098*); the instruct models show large drift on `ifeval` (e.g. olmoe* 0.628/0.731*; the darkest cells) and moderate drift on `bbq`/`gpqa`/`mmlu_pro`. The ‡ marks instance-level fallback. Source: 07-14 status deck.]*

Figure 14.1: The OLMo reproduction at a glance. Base-model classic benchmarks reproduce near-exactly; instruct-model `ifeval` shows the largest drift — consistent with the chat-template-version (`add_generation_prompt`) and fp32-precision findings of Chapter 11 being most consequential on long-form, instruct evaluations.

14.3 Correctness fixes that changed the numbers

Several late fixes are worth recording because each one is a way the aggregate story could have been *silently wrong* — and each is a lesson about honest reporting:

- **Instance-level fallback for CoT benchmarks.** `ifeval`, `gpqa`, and `mmlu_pro` are chain-of-thought scenarios whose core metrics are *instance-only* (`ifeval_strict_accuracy`, `chain_of_thought_correc` HELM emits no run-level aggregate, so those benchmarks were silently *absent* from the drift plot. The fix widens the data contract: fall back to the mean of the per-instance 0/1 scores (which *is* the accuracy), tagged with a ‡ so the provenance is visible. Widening the contract beats hacking the plot.
- **Metric-key granularity trap.** Run-level cells are keyed by full HELM stat descriptions ("`f1_score test on narrativeqa`") while instance-fallback cells are keyed by bare ids ("`ifeval_strict_accur` so the curated headline map (bare names) silently failed to match run-level cells and every run-level benchmark fell to alphabetical-first. The fix normalizes to the metric *family* at the matching boundary and selects the representative on the benchmark's HELM `main_split` (test vs valid differ, e.g. narrative_qa f1 0.597 test vs 0.661 valid).
- **Stale-local dedupe.** An `olmo-7b`/mmlu cell showed a spurious $\sim$0.15 aggregate gap (public 0.295 vs local 0.144) despite near-exact instance agreement. The cause was not drift: every olmo-7b subject directory carried *two* `official_vs_local` pairs — a fresh run plus an un-demoted stale 0.0 attempt — so micro-averaging halved the local mean (124 rows instead of 62). The fix restricts the accumulator to the canonical (first) comparison per report, with a warning per dropped attempt so a future genuine multi-local case surfaces instead of vanishing.

14.4 The shared-gateway route registry

A final infrastructure fix addressed a real operational failure: multiple runbooks (olmo, qwen, gpt-oss, classic-together) share one standing `infer-stack` stack but ship disjoint catalogs, and a converge under one catalog stripped another runbook's still-live routes from the LiteLLM gateway — presenting as a `400 Invalid model name` and (because the minutes-long readiness wait silently never passes once a route is stripped) as the operator's known "failed to acquire lease" events. The fix persists an append-only *route registry* in shared state; every converge merges the invoking catalog with all live deployments and renders the gateway route table from the *whole* registry, so the rendered config is a function of accumulated shared state rather than of which runbook invoked converge — byte-stable renders, so the gateway is never recreated. The reusable insight: *when a shared artifact is rebuilt by multiple writers, render it from accumulated shared state, never from the invoking writer's worldview.*

Chapter 15

Codebase Stewardship

Commits: 2026-07-02, 2026-07-06, 2026-07-12. Three audit/simplification passes interleaved with the research work.

Reproducibility is a property of software, so the health of the codebase is part of the scientific claim: a reviewer who cannot read the pipeline cannot trust its verdicts. Three passes are worth recording as evidence of the "code a researcher can trust" standard.

Correctness audit (2026-07-02/03).
A full codebase audit produced a phased fix plan (P0/P1/P2 findings) implemented over ∼35 commits: e.g. matching/joining on canonical keys rather than raw strings; correcting a perturbed/unperturbed agreement split; filtering non-finite scores at row construction; failure-taxonomy honesty (colors, categories, ordering); content-addressing generated `model_deployments.yaml` filenames; tightening the HF-token permission window. Direct dependencies were declared and `transformers` pinned below 5 (a version whose `is_offline_ mode` change breaks the HELM→EEE rebuild).

Simplicity audit (2026-07-06).
An explicitly orthogonal "reduce bloat" pass: ∼10,600 lines deleted (dead POC/oneoff trees, generated manifests, shims), `helm/diff.py` halved (1,844 → 844), `PRESET_CONFIGS` moved from a 1,154-line dict to a YAML resource, the planning-doc set consolidated (22 → 11 live docs), and the legacy agreement half of the comparison core finally retired (R-2), completing the migration onto `NormalizedDiff`. The methodological gem: pure-relocation refactors of plot-emitting code were proven correct by first *characterizing the noise floor* (render `HEAD` twice, diff, and treat that file set as the only acceptable before/after diff — necessary because plotly emits random div UUIDs).

Fourth simplification pass (2026-07-12).
A post-era "finish what's in flight" pass: Batches 1–3 landed in 22 commits with the full `-run-slow` suite green (638 passed); `helm/` shrank 3,336 →∼ 1,780, `build_reports_summary.py` finished its split (1,715 →∼ 330). A recurring review discipline shows through: audit- agent claims about *why* code exists ("kept for tests") were the ones to re-verify by grep — liveness claims were reliable, purpose claims less so.

Two standing engineering disciplines recur across all three passes and are worth lifting into the paper's reproducibility-methods note: (1) capture a characterization baseline *before* any behavior-changing refactor and gate on `atol`$= 10^{-9}$ equivalence; and (2) submodule hygiene — land changes in the submodule branch first, bump the gitlink as a deliberate separate commit, never auto-commit a dirty gitlink.

Chapter 16

Thread B: Efficient Benchmarking

Repositories: `mrmr_eval` *and* `dkps` *(sibling repos, outside* `eval_audit`*); active ∼2026-06-01 through 2026-06-24; status decks 06-02, 06-09, 06-16, 06-23. Paused thereafter in favor of the HELM-reproduction thread.*

Running in parallel with the reproduction work for the first month was an exploratory statistics thread on *efficient benchmarking*: given that evaluating a model on a full benchmark is expensive, can a small, well-chosen *coreset* of questions plus a regression predict the model's full-benchmark score, and does a Data-Kernel Perspective-Space (DKPS) representation help? The experiment code lived in two sibling repositories forked from the relevant prior work; the LoRA weight-space idea from the literature review (Chapter 4) remained a proposal and produced no code.

16.1 The two framings being compared

16.1.1 Feature selection + regression (Bowyer et al.)

Following *Efficient Benchmarking Is Just Feature Selection and Multiple Regression*, the setup is a binary score matrix (models × questions). Each question is a feature vector in $\mathbb{R}^M$ (M source-model scores) and the target is each model's overall accuracy. One *selects* an informative coreset of questions — primarily with **mRMR** (Minimum-Redundancy-Maximum- Relevance, in MIQ = relevance/redundancy and MID = relevance−redundancy variants) — and *predicts* the full score with ridge or polynomial kernel-ridge regression. The `benchpred` registry implements ∼100 method keys spanning mRMR (ridge/kernel heads), IRT (`pirt`, `gpirt` = Gaussian-process IRT), anchor points, Lasso, and random-sampling/search baselines, each optionally "backfilled" (append `+`) by re-fitting its coreset with ridge/kernel-ridge.

16.1.2 DKPS: a model-centric, transductive predictor

DKPS (Data-Kernel Perspective-Space) is the model-centric alternative. Given a pool of source models with known scores and a target model, all identified only by their *raw responses* to an aligned set of N queries, the method: (1) embeds every (model, query) response (via a text-embedding API, or a one-hot encoding for multiple-choice tasks); (2) forms an inter-model "data kernel" — a models×models Euclidean distance matrix on the flattened per-model response embeddings, normalized by $\sqrt{N}$; (3) computes the *perspective space* by classical MDS on that distance matrix (default 8 components); and (4) fits a linear regression from source coordinates to source scalar scores and reads off the target's predicted score. It is *transductive* — the target's responses participate

in building the MDS space, so the space is re-fit per prediction — and it requires every model to answer the exact same aligned queries under the same embedder.

16.2 Experiment design

The efficient-benchmarking sweeps concentrated on the HELM tasks that carry raw responses (so DKPS is applicable): `legalbench`, `math`, `med_qa`, `wmt_14`, plus `arc_challenge`, `ifeval`, `commonsense`. The knobs swept were: number of source (reference) models $\{20, 30, 40, 50\}$ (sometimes 60); coreset size $\{1, 2, 3, 4, 5, 10, 15\}\%$; three seeds per cell. The reported metrics were MAE, RMSE, Kendall τ, and Spearman ρ (Figure 16.1).

[Figure: `math` — `mrmr_eval` tutorial. A 2×4 grid of RMSE(%), MAE(%), Kendall τ, Spearman ρ; top row versus coreset size (5/10/15%), bottom row versus number of source models (20–50). Methods overlaid: mRMR MIQ (k=5), gp-IRT, Search+, random_sampling_and_learn, Random, AnchorPts, Lasso. mRMR/kernel-anchor methods lead on RMSE/MAE; Lasso is the clear worst. Source: 06-16 status deck.]

Figure 16.1: Representative efficient-benchmarking result on the `math` task. Coreset-selection methods (mRMR, kernel anchor points) substantially outperform Lasso and improve with coreset size; DKPS-family predictors are competitive only in specific regimes (many source models, tiny coresets).

16.3 The DKPS+mRMR active-selection method

The internship's own methodological contribution to this thread was an *adaptive* coreset selector, `dkps_mrmr` (the realization of the "kNN-mRMR online coreset selection" proposal), building the coreset iteratively and *per target model*: seed with one global mRMR pick; then repeatedly (a) DKPS-embed source $\cup$ {target} on the current coreset, (b) take the k source models nearest the target in DKPS space, and (c) restrict the score matrix to those k rows and take one greedy mRMR step to append the next question; finally predict with a standard DKPS transductive regression. It subclasses both `MRMRPred` and `DKPSPred` to reuse the mutual-information machinery and the embedding/regression, and required a new runner capability so `predict` receives the full response matrix rather than a pre-sliced coreset.

16.4 Findings

The concrete results (chiefly the `med_qa` report) are candid, and their honesty is itself worth preserving:

- **DKPS-family methods are not general winners.** At ∼20 source models and ∼50 queries, the feature-selection-paper methods (kernel anchor points, Search+, kernel-mRMR) outperform DKPS and `dkps_mrmr`; on the 40-source-model `med_qa` row, `dkps_mrmr` wins no coreset-size column.
- **DKPS scales with source models where others do not.** DKPS accuracy improves as the number of source models grows, which is not necessarily true of the other methods — and this is where `dkps_mrmr` shines: at a 1% coreset its `med_qa` MAE improves sharply from 9.06 to 5.19 as sources go $20 \to 50$, making it the *single best method at 50 source models* in that corner.

- `dkps_mrmr` **plateaus and is worst at large budgets.** It peaks early (MAE $\sim$4.6%, $\rho \approx 0.93$) and, unlike every other method, does not keep improving to 2.0–2.9% MAE at 10–15% coresets.
- **Cost and representation caveats.** Because the coreset is rebuilt per target, one trial costs $\sim$3 s (1%) to $\sim$2.5 min (15%), versus sub-second for plain DKPS. mRMR is *extremely slow* on continuous-score benchmarks (`wmt_14`). And a one-hot response encoding (used when text embeddings are unavailable) exposes very limited information about a model compared to a text encoding — for `med_qa`, with 5 choices but imperfect answer formatting, the one-hot dimension exceeds 5 and the representation is weak.

16.5 What stayed a proposal

Three lines from the "directions" slides were never implemented and are recorded as open future work: *variance-based early stopping* for adaptive benchmarking; *ensemble* predictors mixing feature-centric (AEA, Scales++) and model-centric (DKPS, Fluid) methods, e.g. re-weighting by DKPS distance from a reference model; and the entire *LoRA weight-space* (W2T) program (SVD canonicalization $\rightarrow$ transformer embedding $\rightarrow$ attribute classification / performance prediction / adapter retrieval, and the exploratory idea of learning a distribution over LoRA weights). This thread effectively concluded at the end of June 2026 when attention returned fully to HELM reproduction.

Chapter 17

Synthesis: What Is Reproducible, and Why

17.1 The headline scientific claims

This section was revised on 2026-07-15 (see Appendix D) to scope its claims to the evidence actually in hand. The original draft asserted a positive result — "a runnable subset of HELM is *reproducible, with residual drift that is small, concentrated, and attributable." That is a plausible and motivating* hypothesis*, but the experiments that would establish it (a frozen population, freshly regenerated stores, end-to-end OLMo confirmation on held-out instances, completed Qwen/classic runs, cross-machine measurements, and a controlled ablation) are not all done. The claims below are the defensible ones.*

The more defensible thesis — and the more scientifically interesting one — is that *reproducing an* LLM *benchmark is a layered system-identification problem.* A public recipe is not a complete experimental record; the producing measurement instrument spans the recipe, the model deployment, the software stack, hardware-sensitive numerical behavior, and the artifact-processing history. Controlled reconstruction can *attribute* and sometimes *close* apparent reproducibility gaps; but when provenance has been lost or transformed, the original experiment can be *non-identifiable from the surviving evidence examined.* What the internship established, at the evidence level actually reached:

1. **The unrecorded execution substrate is real and consequential (established).** HELM records the recipe and the model name but not the dtype, revision, tokenizer/template version, attention kernel, or software-stack versions (of 148 HF-client deployments, 19 pin dtype, 7 pin revision; Appendix B). Individual substrate parameters are *identifiable to varying degrees*: the loader's dtype default and the deployment client class are readable from source/config; the tokenizer's special-token append is readable from the repo; the exact `transformers`/`torch`/CUDA versions and hardware are not.

2. **Attribution pilot on OLMo (partly established, partly pending).** On a small ifeval sample, `float32` candidates best match the sampled public OLMo outputs, consistent with the deductive dtype-default argument (§11.4); the OLMo-7B prompt-independent gibberish is attributable to a tokenizer EOS-append; OLMoE-instruct disagreement is attributable to `add_generation_prompt` chat-template drift. These are *probe-* and *mechanism-level* attributions on a pilot, **not** yet end-to-end HELM reproductions confirmed on held-out instances (the "confirm" step remains the top open experiment; §14 and the paper plan). "Precision is *the* dominant variable" is a hypothesis supported within this pilot, not established across benchmarks or families.

3. **A recoverable-vs-non-identifiable contrast (the strongest conceptual result).** OLMo is the *partially recoverable* case: the missing provenance (dtype, tokenizer, template) can be reconstructed and its effect measured. The original-paper classic models (GPT-J, GPT-NeoX, OPT) are the *non-identifiable* case: their run-specs name a class path that resolves in no released HELM and triangulates to unreleased pre-v0.1.0 code (G13, §12.5), so the producing instrument cannot be uniquely identified or rebuilt. This contrast — some lost provenance is experimentally recoverable, some is fundamentally underdetermined — is a sharper contribution than a uniformly positive "we closed the gaps" story.

4. **Non-runnable cases are a separate gate, not a reproducibility verdict.** Roughly half of the original broad grid could not be *run at all* (gated datasets, revoked download access, missing credentials, packaging, resource contention). These are *filtering/eligibility* reasons upstream of any disagreement measurement — they answer "can we run this?", not "did it reproduce?" — and must not be folded into a negative result.

17.2 The reusable methodology

Independently of the specific numbers, the internship produced a reusable methodology for reproducibility auditing that the paper can present as a contribution:

- **Replay the resolved recipe, never reconstruct it** (Chapter 10): from-spec replay of `run_spec.json` eliminates a whole class of silent reconstruction drift.
- **Freeze the measurement instrument by era** (Chapter 12): version-pinned harness containers make the scoring code a controlled variable, separable from the model.
- **Sweep, don't guess** (Chapter 11): for unrecorded execution parameters, an empirical sweep with feasibility-pruning (but not relevance-pruning) is the honest analogue of matching a recipe.
- **Classify every disagreement** (§3.2): the recipe/environment-vs-reproducibility distinction, enforced through typed exclusion reasons, honest failure-taxonomy colors, and `comparability_unknown:*` signals rather than silent fallbacks.
- **Report a distribution, not a point**: same-machine, cross-machine, and official-vs-local agreement, with per-metric tolerance sweeps, so that "reproducible" is quantified rather than asserted.

17.3 The provenance recommendation for the field

The clearest actionable output for the broader community is that HELM (and any benchmark harness) should record the *execution substrate*, not just the recipe and the model name. Three fields — `model_revision` (commit SHA), `torch_dtype`, and `transformers_version` — transitively close the majority of the reproducibility gap, because the revision fixes the tokenizer/template/generation-config state and the transformers version fixes the precision default and the chat-template behavior. These belong in a machine-readable provenance block (the existing `recipe_facts` slot suffices with no schema change), kept *out* of the run-spec identity string so they do not break official↔local pairing (Appendix B).

17.4 Per-parameter identifiability map

The evidence suggests that identifiability is not a binary property of an entire run. It can be assessed separately for each latent execution parameter.

Parameter	Status in studied cases	Basis and limitation
Benchmark recipe / run spec	Often identifiable	Frozen public `run_spec.json`; artifact migrations can still alter interpretation.
Client / deployment class	Often identifiable	Recorded deployment name and HELM configuration/source; external hosted internals remain opaque.
Load dtype	Partly identifiable	Explicit when pinned; otherwise inferable only conditional on model-loading code and package-version bounds.
Tokenizer special-token behavior	Often recoverable	Repository/configuration inspection and token-level prompt comparison. Exact historical tokenizer revision may remain absent.
Chat template / generation prompt	Often recoverable	Revisioned tokenizer files or controlled rendering sweeps; effective behavior depends on package version.
Model/tokenizer revision	Frequently unidentified	Rarely pinned in HELM metadata; aliases and mutable repositories prevent unique historical recovery.
Inference engine and version	Partly identifiable	Client family may be known; exact serving version, kernels, scheduler, and generation defaults often are not.
Batching, cache, topology, numerical flags	Usually unidentified	Not represented in public artifacts and often provider-specific.
Hardware / CUDA / driver	Usually unidentified historically	Can be recorded and robustness-tested prospectively, but exact historical replay may be impossible.
Harness era	Recoverable for tagged runs; proxy-only for some classics	Tagged release commits support era containers. Migrated paths can fingerprint an unreleased origin without yielding a buildable exact instrument.

17.5 Open threads at the close of this chronology

Several items were scoped or built but not fully landed, and are the natural next steps for the paper's experiments:

- The HuggingFace-in-process *routing switch* — the mechanism exists (reserve-only lease + fp32 `HuggingFaceClient`), but the default replay path does not yet auto-route a HuggingFaceClient official to it.
- The RedPajama era path had local run/report evidence by July 14, but those `/data` artifacts were excluded from the supplied archive. The Qwen, GPT-OSS, classic GPT-J/GPT-NeoX/OPT, and ordinary-path OLMo confirmation status still required explicit artifact-level

verification.

- Regenerating the stale on-disk OLMo report store so the corrected drift plots (instance-level fallback, metric-family resolver, stale-local dedupe) populate the published artifacts.
- Pinning model revisions through `eval_audit`'s bundle boundary (a ∼3–4 file change) so generated bundles carry the SHA automatically.

Appendix A

The HELM Gotchas Catalogue (G1–G13)

This appendix condenses the running ledger of undocumented or hard-to-see HELM behaviors accumulated while building the audit pipeline (`docs/helm-gotchas.md`). Each is a concrete, reviewer-anticipating pitfall that a reproducibility paper's appendix should address.

G1. `run_spec.json` schema evolves silently between releases.
Newer HELM populates `adapter_spec` fields older HELM omitted (`chain_of_thought_prefix`, `global_suffix`, `num_trials`, `model_deployment`, top-level `metric_specs` / `groups` / `annotators`), so byte-for-byte hashing yields $0/N$ matches across releases even for identical recipes. Workaround: a schema-collapsed *canonical recipe hash* (§14.1.1).

G2. The local `suite` field is the experiment name, not a public-track version.
A versioned join on `logical_key + suite_version` yields 0 matches; the funnel detects a non-version-shaped local suite and reports the versioned join as not meaningful (`N/A`) rather than 0.

G3. The `huggingface/<model>` deployment alias does not always exist.
Many open-weight runs were executed through the HF API, but HELM's registry ships deployment YAMLs for only a subset; a missing alias forces a different stack (vLLM/etc.), introducing deployment-substitution drift. Workaround: register a custom local deployment.

G4. `model_deployment` semantics differ across versions.
Older HELM did not record the field (defaulting to "the HF API"); newer does. A local `huggingface/<model>` and an official `<MISSING>` are semantically identical but hash differently — schema-evolution, dropped from the canonical hash.

G5. `adapter_spec.instructions` differs between releases on identical scenarios.
MMLU official prepends "Answer with only a single letter."; several LegalBench scenarios prepend a list-the-allowed-labels preamble. This is *genuine* recipe drift (a different prompt), surfaced via the `same_instructions` fact — not to be silently normalized. (The BBQ `output_format_instructions=mcqa` drift of §8.4 is the same species.)

G6. The `classic` corpus moved bucket prefixes.
Classic runs migrated from the bucket root to `classic/benchmark_output/runs/<ver>/`, mirroring every other suite; a special-case in the downloader was removed.

G7. Same weights $\neq$ same output across serving stacks.
Even greedy, HF `generate()` and vLLM differ in kernels, tokenizer batching, stop-sequence

handling, unspecified sampling defaults, and EOS/pad handling — a 3–6% per-instance flip rate on multiple-choice, ∼30–40% on long-form. Mitigations: match the original client, pin decoding, or document as `deployment_drift`. (Chapter 11 is the deep dive.)

G8. `metric_specs` schema evolution.
Top-level `metric_specs` differ on the majority of packets from restructuring, not a recipe change; excluded from the canonical hash — compare the metric *output*, not the declaration.

G9. Local index `model_deployment` is audit-specific.
Custom local deployment names (e.g. `kubeai/vicuna-...-no-chat-template-local`) do not appear in the public run spec; the logical-key join collapses across deployments and `same_deployment` correctly reports `no`.

G10. Public-track `suite_version` is not the HELM release version.
The same `run_spec.json` can appear identically in `v0.2.4` and `v0.3.0`; use `run_spec_hash` to detect recipe-identical duplicates across suites. (The era-replay containers key the *instrument* on `(public_track, suite_version)`, which is the right granularity for the classic track — see Chapter 12.)

G11. `per_instance_stats.json` corruption on giant runs.
Certain msmarco runs were truncated mid-write during upload (consistent byte-offset `JSONDecodeError`); recently re-uploaded versions parse cleanly. Redownload on a size mismatch.

G12. `run_spec_hash` canonicalization differs by release.
HELM's own canonicalization is version-dependent, so semantically identical specs hash differently across releases; the coverage-side canonical hash applies a stable normalization on top.

G13. Classic-corpus `class_name` paths resolve in *no* HELM version.
The stored `helm.benchmark.basic_metrics.BasicMetric` (helm-prefix + flat) is a once-migrated hybrid — a naive `benchmark.`→`helm.benchmark.` rewrite of a still-flat production path — pinning the producing code to unreleased pre-v0.1.0 HELM (2022-07-31 to 08-26). Class-path *shape* is the origin fingerprint; fixed by a self-verifying declared class-path canonicalization in the era shim (§12.5).

Appendix B

The Unrecorded-Parameter Taxonomy

Condensed from `docs/helm-unrecorded-deployment-params.md`. HELM serializes the recipe and the model's name but almost nothing about *how* the numbers were computed. Empirically, of 148 HuggingFaceClient deployments in HELM's `model_deployments.yaml`, only **19 pin** `torch_dtype` (all `bfloat16`) and only **7 pin a model** `revision`. The parameters, ranked by effect on a greedy (temperature-0) output:

Tier	Parameter	Why it matters
1	Model weight revision (SHA)	Same name ≠ same weights; authors re-upload checkpoints/-configs/tokenizers. The single highest-leverage field — fixes the tokenizer, chat template, and generation config as-of-then.
1	Load precision / dtype	Unpinned ⇒ transformers-version-dependent default (pre-v5 = float32). *Pilot evidence*: HF fp32 exactly matched the sampled OLMoE outputs (§11.4).
1	Quantization	GPTQ/AWQ/fp8/bnb change the weights outright.
1	Software-stack versions	The meta-parameter: transformers sets the fp32-vs-bf16 default *and* whether the chat template honors `add_generation_prompt`; engine/flash-attn versions change kernels. Nowhere in the run dir.
2	Attention implementation	eager/sdpa/flash/PagedAttention change reduction order ⇒ different logits at temp 0.
2	Matmul / TF32 precision	For fp32 especially, true-fp32 vs TF32 matmuls differ.
2	Hardware / GPU model	Determines kernels, TF32 behavior, whether fp32 MoE fits.
2	Device topology / TP	Multi-GPU all-reduce order differs from single-GPU.
2	Batch composition	Kernels are not batch-invariant; same prompt, different batch neighbors, different logits.
3	Tokenizer + chat-template version	The `add_generation_prompt` drift (§11.3); the templated prompt is never stored.
3	`add_special_tokens` / BOS	The OLMo-7B EOS-append case (§11.1).
3	`generation_config.json` defaults	Injects unspecified fields (repetition penalty, eos/pad ids).
3	Stop-sequence / detokenization	HF stops on token ids, vLLM on string match ⇒ different truncation and scored text.

Tier	Parameter	Why it matters
4	RNG seed; KV-cache dtype	Mostly for sampling (temperature > 0) and serving-engine reproductions.

The one non-obvious lever. Several Tier-3 gaps collapse into Tier-1: the tokenizer, chat template, and generation config all live *inside* the model repo, so pinning the model revision recovers all of them as-of-then. `model_revision` + `torch_dtype` + `transformers_version` close the majority of the gap. Both weight-loading engines already accept `revision` as a data-only edit (infer-stack `catalog.yaml`; HELM's `client_spec.args`); the only code gap is that `eval_audit`'s bundle boundary drops it (a $\sim$3–4 file fix). These fields must live in the `recipe_facts`/provenance block, *not* the run-spec identity string, or a SHA in the pairing key would break official$\leftrightarrow$local pairing.

Appendix C

Deliverables and Timeline

C.1 Internship timeline at a glance

Dates (2026)	Work
May 15 – Jun 2	Onboarding; literature review (LoRA/W2T, efficient benchmarking). Thread B begins.
Jun 3 – Jun 4	First commits: e2e adapter config and tests (Chapter 5).
Jun 9	Corpus cartography / filter statistics (Chapter 6).
Jun 11 – Jun 12	Repo refactor (Phases 0–2) and the unified comparison core (Phase 3 / `NormalizedDiff`, judge registry) (Chapter 7).
Jun 12 – Jun 18	OLMo campaign begins: tokenizers, chat-template window budget, the 114-packet canonical-key fix (Chapter 8); Thread B `dkps_mrmr` lands (Jun 24).
Jun 16 – Jun 24	Containerized HELM execution and the infer-stack leasing pipeline (Chapter 9).
Jun 25 – Jun 30	From-spec replay: run-spec verbatim replay, deployment rewrite, rel-path addressing, content-addressed caching (Chapter 10).
Jun 30 – Jul 10	Deployment matching: OLMo-7B EOS-append, the deployment-match tool, `add_generation_prompt` drift, the float32 discovery, the unrecorded-parameter taxonomy (Chapter 11).
Jul 2 / 6 / 12	Three codebase audit / simplification passes (Chapter 15).
Jul 8 – Jul 13	Era-pinned containers, the shim, redpajama-3b runbook, classic Together models, the G13 archaeology (Chapter 12).
Jul 11 – Jul 14	GPT-OSS-20B and Qwen fan-outs (Chapter 13); aggregate score-drift reporting fixes, route registry (Chapter 14).

By commit count: 452 authored commits, distributed as 181 in June and 271 in July; the busiest days were 2026-07-06 (50 commits, the simplicity audit) and 2026-07-12 (39, the fourth simplification pass).

C.2 Key artifacts produced

Reproduction runbooks (`reproduce/`, `dev/`): `olmo_models_combined`, `gpt_oss_20b_from_spec`, `qwen_models_combined`, `classic_together_combined`, `dev/e2e-tests` (phi-2), `dev/era-tests` (redpajama-3b).

Tools and libraries : `dev/tools/deployment_match/` (the serve-and-probe deployment matcher with `hf-probe`, `hf-match`, fp32-TP, and `compare_prompt.py`); `docker/era_shim/` (the standalone era from-spec replay CLI + OpenAI-compatible client); `eval_audit/eras.py` and `docker/eras.yaml` (era registry); `eval_audit/hf_inprocess.py` (reserve-GPU in-process reproduction); `eval_audit/normalized/diff` (`NormalizedDiff`, the unified comparison core); `eval_audit/judge_registry.py` and `eval_audit/metrics_taxono`

Documentation (the intended paper appendix material): `helm-gotchas.md` (G1–G13), `vllm-vs-huggingface-de` `helm-unrecorded-deployment-params.md`, `eee-vs-helm-metadata.md`, `pipeline.md`, `architecture.md`, `container-execution.md`, and the 19 planning documents under `docs/planning/`.

Thread B (sibling repos): `mrmr_eval` (the `benchpred` efficient-benchmarking harness with the `dkps`/`dkps_mrmr` methods and the `med_qa` report) and `dkps` (the vendored DKPS library and HELM example runners).

C.3 A note on method and provenance of this document

This chronology was reconstructed after the fact from the primary sources listed in Chapter 3. Where numbers are quoted (agreement ratios, token counts, corpus statistics, the fp32 sweep table, the efficient-benchmarking results) they are taken directly from the status decks, the developer journals, or the committed reports; where a claim is a design decision or a lesson, it is drawn from the corresponding journal entry or planning document. The development was carried out with heavy use of an AI pair-programming harness (recorded in the first-person developer journals); the research direction, design decisions, GPU-host execution, and interpretation are the author's. Readers preparing the academic manuscript should treat the §-cross-referenced figures and tables here as pointers into those primary artifacts, which remain the ground truth.

Appendix D

Interpretive Revision (2026-07-15): Epistemic Status of the Central Claims

The body of this chronology reconstructs, in a single after-the-fact narrative voice, what was believed and reported across the internship. Some of those claims — especially around the `float32` result — were stated more strongly than the evidence currently supports. Rather than rewrite the dated narrative (which would erase what was believed at each point), this appendix records the corrected epistemic status in one place. It was prompted by an external review of the chronology and the paper plan, and its conclusions are folded into `docs/planning/tmlr-paper-plan.md`.

D.1 A workflow-status vocabulary

Claims in the body should be read against *where in the pipeline* their evidence sits. These are not interchangeable, and the body sometimes chose the strongest available interpretation:

1. **Infrastructure implemented** — code exists, unit-tested on a CPU host.
2. **Smoke-tested execution** — ran end-to-end on a tiny input, once.
3. **Deployment-match probe** — an oracle-scored config sweep over a small sample, possibly using non-default request knobs. *Discovery, not confirmation.*
4. **Exact-spec run** — the official `run_spec.json` executed verbatim on the real path.
5. **Complete HELM artifact generation** — normal `scenario_state.json`/`stats.json` produced.
6. **Final comparison report** — run through the standard pair-report pipeline.
7. **Fresh vs. stale store** — regenerated with current code vs. a store predating a relevant fix.

The single most important correction: the `float32` result (§11.4) sits at level 3 (probe / discovery), not levels 4–6. The "confirm" step that would move it to a reproduction result — land the winning config on the ordinary HELM path, run the exact public `run_spec.json`, generate normal artifacts, compare via pair-report, and evaluate on *held-out* instances — has not been run for any model. Configuration discovery and held-out confirmation must be separated, or a config fit to a small sample is reported as if it were an identified fact.

D.2 A finer disagreement taxonomy

§3.2's binary — "recipe/environment failure" vs. "true reproducibility failure" — is too coarse, and the phrase "true reproducibility failure" should be *avoided* whenever execution provenance is

incomplete. Keep the upstream run-vs-not-run gate (non-runnable = a filtering/eligibility reason, not a disagreement), and for observed disagreements use:

1. **Recipe mismatch** — the prompts/decoding/scenario differ (e.g. the BBQ `output_format_instructions` drift, G5).
2. **Deployment mismatch** — serving engine / client class differs (vLLM vs. hosted API; `HuggingFaceClient` vs. vLLM).
3. **Execution-instrument mismatch** — precision, tokenizer/template version, attention kernel, device placement, software-stack versions.
4. **Artifact-interpretation or migration mismatch** — the pandas row-order / instance-id case (EEE Case A); the G13 class-path migration.
5. **Residual disagreement under controlled equivalence** — the only category that resembles conventional repeatability failure: everything controllable is matched and a gap remains.
6. **Historically non-identifiable reproduction** — the surviving artifacts and releases do not uniquely specify what faithful reproduction would even mean (GPT-J / GPT-NeoX / OPT via G13). Qualitatively different from (5): not "it failed to repeat" but "the target is underdetermined."

Most of what the body called "deployment-boundary artifacts" are categories (2)–(4) — recoverable by controlling the substrate. Category (6) is the negative counterpart and is a first-class result, not a gap to be closed.

D.3 Corrected-claims table

As stated in the body	Corrected status
"The official OLMoE run — and every OLMo HF-client run — executed in `float32`."	The *loader code path* defaults to fp32 under every `transformers` version consistent with the run date (a deductive claim, the stronger leg), *conditional* on the unrecorded `transformers`<5. Likely, not proven.
"fp32 reproduces the official completions *exactly*."	Among tested configs, fp32 candidates *best matched the sampled* public outputs ($n \approx 12$ ifeval), using a probe-only `add_special_tokens=False` knob. Not an end-to-end HELM reproduction; not held-out.
"all four officials reproduce at fp32."	Probe-level, ifeval-only, instruct-only. OLMoE via HF fp32; dense OLMo-2 via vLLM fp32; the HF probe does *not* reproduce OLMo-2 (an open execution-sensitivity finding, below).
"Precision is the dominant hidden variable."	Supported *within* the OLMo ifeval pilot; not established across benchmarks or model families.
"the OLMo-2 HF divergence is the probe mis-generating, not science."	Reframed: a *current* HF stack failing to reproduce a historical result *recorded as a `HuggingFaceClient` deployment* under byte-identical prompt tokens is itself an unresolved systematic divergence under prompt-token equivalence — evidence for the thesis (with a dense-model probe artifact still on the candidate-cause list).

As stated in the body	Corrected status
"residual drift is small, concentrated, and almost always attributable."	A motivating *hypothesis*, pending the confirm step, fresh stores, cross-machine data, and a controlled ablation. Not yet established.
OLMo/GPT-OSS heatmaps "exist".	*Split by store (collaborator-verified 2026-07-15).* GPT-OSS report files were regenerated Jul 14 at 13:47 after reporting fixes, but the run and comparison provenance must still pass the preservation and acceptance gate. `olmo-models-combined` is stale: the reported on-disk headline still shows the un-deduped `olmo-7b` halving (MMLU **0.295/0.144**, GSM 0.036/0.018, narrative_qa 0.597/0.311), whereas the provisional corrected value cited in the figure is 0.295/0.287. Regenerate before citing any OLMo base-model number.

None of these corrections weaken the *methodological* contributions (from-spec replay, era-pinned instruments, sweep-don't-guess attribution, the provenance taxonomy) or the *conceptual* contribution (recoverable vs. non-identifiable provenance). They scope the *empirical* claims to what has actually been run.

Appendix E

Collaborative Claim–Evidence–Status Matrix

Status note. This appendix is a human-readable snapshot. The canonical, machine-readable source is `chronicle/data/master_claim_evidence_ledger_2026-07-15c.csv`; update that file first and regenerate this table before submission.

Claim	Status	Qualification / principal evidence
Name-only matching can misclassify comparable and incomparable runs.	**Established**	Early `groups=` and omitted-temperature cases; exact run specs later became the authoritative recipe source.
The inherited EEE study detects gaps but does not attribute the residual substrate.	**Established**	NeurIPS Case Study 3 and its appendix; Pythia/Vicuna/Falcon scope predates the internship.
Exact public `run_spec.json` replay removes observed prompt-reconstruction drift.	**Established**	BBQ output-format instruction case and the June 25 from-spec implementation.
OLMo-7B prompt-independent gibberish was caused by appended special-token behavior.	**Established**	Token-level prompt diagnosis and tokenizer override.
IFEval output is sensitive to chat-template and `add_generation_prompt` behavior.	**Established**	Controlled prompt rendering and deployment-match sweeps.
Float32 candidates best matched sampled OLMo instruct outputs in the 12-instance IFEval probe.	**Preliminary**	Discovery-set oracle ranking. The ordinary HELM path and held-out confirmation remain open.
The public OLMoE load most likely used fp32.	**Preliminary**	Conditional source-level inference: unpinned dtype, HuggingFaceClient path, bounded Transformers 4.x behavior. Exact historical package environment is absent.

Claim	Status	Qualification / principal evidence
The dense OLMo-2 current-HF divergence has a known single cause.	**Unresolved**	Prompt-token mismatch was rejected; model/tokenizer revision, software, attention, placement, decode behavior, and numerical details remain candidates.
RedPajama-3B ran through both later classic proxy eras.	**Locally supported; not packaged**	Journal/slide and report-refresh references support local completion; raw `/data` reports are not included in this source archive.
GPT-J/GPT-NeoX/OPT public artifacts identify a unique released producing harness.	**Unresolved**	Evidence supports the opposite: migrated class paths point to an unreleased source interval and later v0.2.4/v0.3.0 images are declared proxies.
Public classic artifacts were carried forward between v0.2.4 and v0.3.0.	**Established**	Byte-identical public artifacts for shared runs; not two independent officials.
The compatible classic source interval is July 31–August 26, 2022.	**Established**	Source-history triangulation under the migration interpretation; it is an inferred compatibility interval, not an observed run timestamp.
The complete OLMo/Qwen/GPT-OSS/classic performance grids are publication-ready.	**Unresolved**	Stores and workflows existed at different levels, but freshness, raw artifact preservation, and final exact-path completion differ by family.
A position/systems paper is supportable now.	**Established**	The conceptual model, forensic cases, system implementation, and provenance recommendations are substantial if empirical claims remain scoped.
A broad technical claim that most residual gaps are small and attributable is supportable now.	**Planned**	Requires a frozen denominator, fresh stores, ordinary-path confirmation, held-out validation, ablations, repeatability baselines, and archived artifacts.

Appendix F

Source Inventory and Reproducibility Notes

- Repository archive: `eval_audit-source-2026-07-14T144915-5-b8b858697e7c.tar.gz`; supplied HEAD `b8b858697e7c2491af163c8f5d757fe6ae6f3c1a`.
- Weekly slide archive: `internship-powerpoints.zip`, seven decks from June 2 through July 14.
- Detailed Claude chronology: preserved under `sources/claude_chronology_2026-07-15.tex`.
- Independent evidence-led chronology: preserved under `sources/openai_chronicle_2026-07-15.tex`.
- Machine-readable commit and slide ledgers are included under `data/`.
- The source archive manifest excludes untracked, ignored, and `/data` result stores. Any claim based on those stores must be accompanied by a separately preserved artifact bundle before publication.

Appendix G

Publication-Grade Preservation Checklist

A paper artifact should preserve source commit and dirty state; container digests; Python, Transformers, PyTorch, CUDA and driver versions; GPU model and topology; model and tokenizer commit SHAs; effective dtype, quantization, attention backend, TF32/matmul flags, batching, caching, and tensor parallelism; raw, rendered, and tokenized prompts; complete generation configuration; frozen public run specs and hashes; raw local and public outputs; every deployment-search candidate rather than only the winner; compatibility rewrites; report-generation version; seeds and repeated-run manifests; and all analysis/figure scripts.

Appendix H

Main-repository commit ledger

This appendix lists every commit authored as `Edward Wang <edward.wang@kitware.com>` in the supplied `eval_audit` history from 2026-06-03 through 2026-07-14. Insertions and deletions are Git numstat totals and should not be interpreted as net authored lines: they include refactors, moves, generated fixtures, and regenerated lock files.

2026-06-03 (1 commits)

Hash	Subject	+	-	Files
`ca46b11e`	Add adapter config for the e2e test and update matching logic	319	74	2

2026-06-04 (1 commits)

Hash	Subject	+	-	Files
`ed9e40ed`	Add e2e tests	263	0	6

2026-06-11 (19 commits)

Hash	Subject	+	-	Files
`609e02bb`	Fix stale prioritized-example repair test to match no-regeneration contract	48	34	1
`59fbdc2d`	Add repo refactor plan and journal entry	355	0	2
`f3953650`	Phase 0a: clean up repo root (refactor plan)	18	12	5
`6f294d03`	Phase 0b: finish vllm_service -> infer_stack rename	2	2	3
`3507539d`	Phase 0c: consolidate root docs	5	2	4
`4a0ae5b7`	Phase 1: every console script resolves to eval_audit.cli.*	71	12	9
`6d5035aa`	Phase 1: retire grouped-CLI dispatchers and dead alias module	34	37	7
`4473bfc9`	Phase 1: role docstrings for core_* modules; fix stale runbook command	46	1	6
`b3a4b272`	Phase 2 prep: hoist @profile shim into infra/profiling.py	44	102	15
`7f64806b`	Phase 2: split build_reports_summary into reports/summary/ subpackage	4570	4235	12
`72bfeb3d`	Journal: refactor plan implementation session (Phases 0-2)	85	0	1
`9b7ddb74`	Phase 2: extract Stage 1 library logic from cli/index_historic_helm_runs	639	568	3

Hash	Subject	+	-	Files
ca6f5d7b	Phase 2: extract diff primitives from helm/diff.py	715	668	2
dd742728	Phase 2: split core_metrics into curves / plots / tables modules	2165	2003	5
9a377838	Phase 2: split filter_analysis into tables / text / charts / io modules	1313	1206	5
c201938f	Phase 2: split eee_only_heatmap into data / render modules	1243	1173	3
9fcdfefe	Phase 2: split helm/analysis into instance_stats / analysis_report	1075	1028	3
3822d676	Mark Phase 2 secondary targets done in plan; journal entry	91	6	2
5b05562f	Add Phase 3 design docs (comparison-core unification + equivalence matrix)	522	27	3

2026-06-12 (28 commits)

Hash	Subject	+	-	Files
76a320c3	Revise Phase 3 design docs for the new research program	389	261	3
352d55d0	Journal: Phase 3 design + research-program revision	73	0	1
94c7fb49	Phase 3 / 4.0: lift metric taxonomy out of helm/, add judge-dependence class	283	115	4
4a36e6d4	Phase 3 / 4.1: recipe_facts accessor + judge-identity fields	453	1	3
1836be3e	Phase 3 / 4.2: normalized/diagnose.py – framework-free diagnosis port	559	0	2
6656048b	Phase 3: behavior baseline capture harness + committed F3/F4 snapshots	1919	0	5
426a211a	Phase 3 / 4.4: EEE synthesis library home + zero-HELM EEE entry points	507	436	11
12623aa5	Journal: Phase 3 implementation session (4.0-4.2, baseline, 4.4)	75	0	1
a347eca0	Phase 3 / 4.3: NormalizedDiff – the unified comparison core (additive)	616	102	3
cc6c9e8b	Phase 3: judge-identity inventory across the closed-judge benchmarks	86	0	1
dfe2c941	Journal: Phase 3 continuation (4.3 + judge inventory)	62	0	1
5c8f7a48	Phase 3 / 4.6: route the renderer through NormalizedDiff; single diagnosis impl	59	271	3
9e40aa92	Phase 3 / 4.5 (loader half): declared instance-source policy + F6 probe	206	22	2
e5f905c7	Phase 3 / 4.5 (renderer half): –instance-source flag + recorded provenance	75	0	6
0299ed03	Journal + design-doc status: Phase 3 sub-stages 4.0-4.6 complete	75	0	2
3ef5233e	Phase 3 / 4.9a: curated judge registry + identity resolution	166	3	3
900ab05c	Phase 3 / 4.9b+c: declared judge substitutions through planner + renderer	363	18	5
5b34364f	Phase 3 / 4.9d: Stage-1 –allow-closed-judge-benchmarks relax	147	13	3
ca83c051	Phase 3 / 4.7: draft the upstream EEE recipe_facts issue	89	0	1
f099bb2a	Phase 3 / 4.8: docs pass + legacy-bridge marking + final status	100	27	5
ac3445ee	Journal: Phase 3 completed (4.9 + 4.7 draft + 4.8)	69	0	1
aaa2c928	Fix infer_stack adapter: drop removed root= kwarg from load_profile_contract	6	1	1
bcb0e16f	Add OLMo reproduction smoke suite: adapter presets, run scripts, and virtual experiment config	1385	0	14
ec4960aa	Update deprecated hf auth call	1	1	1
92f9582a	Switch profiles without prompting	1	1	1
31961a84	Fix OLMo tokenizer aliases to match HELM registry	3	3	1
517cbbf5	Add sentencepiece/tiktoken tokenizer backends as deps	6	0	1
2aea115d	Fix olmo-7b tokenizer load via a HELM entry-point plugin	58	6	2

2026-06-15 (4 commits)

Hash	Subject	+	-	Files
f85b1817	Add OLMO_FORCE_RERUN to re-run OLMo smoke grid over prior results	25	1	2
38698653	Add OLMo full-run grid; rename runbook to olmo_models; group on full runs	226	57	13
dd6575b6	Add updated config to work on aiq-gpu	82	12	1
bef5ccc0	Restrict to gpus 2,3	5	0	1

2026-06-16 (7 commits)

Hash	Subject	+	-	Files
5c0490c9	Reserve chat-template headroom for OLMo openai-compatible runs	98	0	2
8cb5eb75	Stage natural_qa data via gcloud auth + helm-run shim (script-only)	363	0	6
01a68880	Drop OLMo math runs (competition_math disabled on HF Hub)	14	7	1
ddc40407	Drop OLMo natural_qa runs (GCS bucket denies authenticated reads)	40	364	7
4354ef68	Default OLMo smoke grid to force-rerun; keep full grid opt-in	10	5	2
5d02e12b	Add containerized HELM execution path for the kwdagger pipeline	1484	7	17
7b028102	Restructure phi-2 e2e tests into the olmo_models runbook shape	746	121	17

2026-06-17 (9 commits)

Hash	Subject	+	-	Files
4d5c411e	Add containerized phi-2 e2e example (vLLM served on host, HELM via –network host)	327	11	10
63f8c8cd	Enable public comparisons for the olmo full virtual experiment	5	5	1
0d4a4183	docs(planner): add robust order-insensitive matching plan	200	0	1
b7373676	feat(run_entries): add order-insensitive canonical_logical_key	147	0	2
a25aac93	feat(planner): group runs by order-insensitive canonical key	252	100	2
d8e6b698	docs(planner): document canonical-key matching + journal entry	91	0	2
ceefb460	Merge branch 'main' into olmo-reproduction	0	0	0
36b7860a	Merge branch 'olmo-reproduction' into main	0	0	0
f46445d9	Merge branch 'main' into e2e-refactor	0	0	0

2026-06-18 (9 commits)

Hash	Subject	+	-	Files
f23bf696	Run the phi-2 container e2e example by default (opt-out)	73	28	4
0e8617a7	Fix dangling venv python in the HELM-runner image (multi-stage uv)	59	1	2
fa37fd6c	Run HF e2e scenario first on a freed GPU; spin vLLM down at grid start	59	7	5
1cb228a5	Enable public comparison in the non-smoke e2e experiments	67	26	3
de439c56	Split the e2e-phi2 report into one virtual experiment per scenario	282	170	11

Hash	Subject	+	-	Files
89695fe6	Force matplotlib Agg backend in report rendering (fix Tk core dump)	68	0	6
55fb5ecf	Merge branch 'e2e-refactor' into main	0	0	0
cc57d5d7	Run OLMo reproduction through the containerized HELM pipeline	430	4	8
c8b3d525	Align OLMo BBQ run entries with public recipe (output_format_instructions)	391	7	4

2026-06-22 (5 commits)

Hash	Subject	+	-	Files
10c3db97	Migrate infer_stack integration to the catalog/leasing API	387	286	5
f779d596	Reschema the e2e + OLMo infer-stack config dirs to catalog.yaml	169	548	8
f4236d61	Migrate the e2e + OLMo runner scripts to the leasing CLI verbs	167	108	8
e8e76818	Update the e2e + OLMo runbook docs for the leasing CLI	73	52	3
ab8af6c8	journal: infer-stack CLI/API migration session	73	0	1

2026-06-23 (8 commits)

Hash	Subject	+	-	Files
4afe6a66	Merge branch 'olmo-docker-pipeline' into main	0	0	0
22395e21	Merge branch 'main' into infer-stack-cli-api-migration	0	0	0
d259d248	docs: plan for infer-stack <-> kwdagger high-throughput integration	499	0	1
dac25978	journal: leasing-pipeline-lifecycle planning + implementation session	132	0	1
ee34857c	Messy state	952	36	19
da930f4c	Remove old-schema config.yaml resurrected by a bad merge resolution	39	162	2
de9919db	docs: drop I4/I5, add eval_audit-integration hand-off	212	28	2
24991d17	Wire per-run infer-stack GPU leasing into the HELM fan-out	960	18	11

2026-06-24 (11 commits)

Hash	Subject	+	-	Files
4158bea0	Require containerized HELM execution; leasing as an orthogonal bracket	358	259	7
c5ff9a35	Containerize the e2e presets; drop the redundant -container scenario	72	129	5
03dc08e5	Lease + containerize by default across the OLMo and e2e grids; drop the toggles	312	292	9
25811227	docs: containerization mandatory + leasing default; session journal	332	105	4
5317e14f	Update gitmodules and bump commit hashes	6	6	5
ac444241	Merge branch 'main' into infer-stack-cli-api-migration	0	0	0
31dda44c	infer_stack: make protocol_mode a required preset fact (no silent chat default)	134	8	3
2fd3fd6c	e2e/olmo grids: pass –yes to the bootstrap 'release –evict'	4	4	4
94980dda	journal: leasing CLI bringup notes (_load_catalog AttributeError fix)	47	0	1
d9f075db	infer_stack: bump submodule to the _load_catalog catalog-field fix	1	1	1
b2aff8fe	infer_stack: bump submodule to the dev/lease-polish merge (995f45e)	1	1	1

2026-06-25 (30 commits)

Hash	Subject	+	-	Files
fde72004	infer_stack: bump submodule to the dev/lease-polish merge (0cde11d)	1	1	1
be2a72a6	catalogs: declare protocol: completions for base-model endpoints	80	7	3
23a20459	e2e: always force-rerun both grids; remove the E2E_FORCE_RERUN opt-out	21	19	2
248c3b4b	e2e: recap every scenario's report path at the end of 40_build_summary	26	0	1
43d3c2b0	e2e: drop the broken "start here" README link from the summary recap	1	2	1
212289e5	docs(planning): bring the run-from-run-spec replay plan onto the impl branch	404	0	1
1cd44778	manifests: add from_run_spec + –precomputed-root to make-manifest	41	0	2
622881b2	pipeline+bridge: route from_run_spec runs to the replay docker node	64	0	2
f54d29c4	test+journal: from_run_spec pipeline wiring tests; session journal	284	0	2
7ee5ecee	submodules: bump aiq-magnet gitlink to the from-spec replay CLI (4b10e1b)	1	1	1
d5180845	submodules: bump infer_stack gitlink + track its working branch	2	2	2
bd097610	gitmodules: track aiq-magnet impl/run-from-run-spec branch	1	1	1
5e1edb4b	docs(planning): plan to migrate the phi-2 e2e to run_spec.json replay	237	0	1
353e74b5	docs(planning): correct the phi-2 e2e from-spec plan after a code review	59	27	1
f18e5036	e2e(from-spec): HF override + sibling from-spec manifests (Change 1)	113	0	3
1d90f453	e2e(from-spec): teach export-benchmark-bundle the from-spec path (Change 2)	78	2	2
d81ca3ca	e2e(from-spec): gate the grid on E2E_FROM_SPEC, carve out incomparable (Change 3+4)	56	10	3
2368fe9a	e2e(from-spec): tests for the phi-2 from-spec exporter + discovery (Change 6)	223	0	1
ea0141ae	journal: session entry for the phi-2 e2e from-spec review + implementation	59	0	1
daf6dfbb	e2e(from-spec): make faithful replay the default, drop the opt-out	81	136	8
33a0606f	docs(e2e): reflect from-spec-as-default (no toggle) in plan + README	84	47	2
05d2932e	journal: from-spec becomes the e2e default (toggle removed)	36	0	1
e2543fb5	fix(e2e from-spec): override dropped phi-2's context window -> window-service crash	15	0	2
05a704d3	docs(planning): plan to record the local deployment on from-spec replays	262	0	1
1a119919	docs(planning): lock in explicit –model-deployment (reject auto-derive)	31	19	1
f65997d3	from-spec(deployment-rewrite): record the LOCAL deployment, unmask same_deployment	449	71	12
b783a95c	docs(from-spec): mark deployment-rewrite implemented; amend superseded plans	165	17	7
a4b5bac5	docker(helm-runner): split helm/magnet install so magnet edits skip the heavy layer	27	6	1
bad2755f	gitignore: ignore docker/build.sh staging context (.build-staging/)	3	0	1
abc00b3c	submodules: bump aiq-magnet gitlink to the –model-deployment rewrite (50489f0)	1	1	1

2026-06-26 (1 commits)

Hash	Subject	+	-	Files
1eb52704	docker(helm-runner): raise build-step nofile ulimit to survive uv bytecode compile	14	0	1

2026-06-29 (19 commits)

Hash	Subject	+	-	Files
8d96a471	e2e(infer-stack): resolve data_dir as env > settings.yaml pin > /data default	54	18	2
0f09fe87	submodules: bump every_eval_ever gitlink to tmp-branch (b1c5a0f0e)	1	1	1
15020825	infer_stack: bump submodule to the dev/lease-polish catch-up merge (045536c)	1	1	1
b5c4cfed	fix(eee-convert): replay run prod_env so local deployment aliases resolve	45	0	1
03b98e9f	feat(e2e): aiq-gpu rsync fetch utilities (17 + 42 + shared lib)	292	1	4
eb904ea7	docs(journal): session entries for aiq-gpu rsync utilities + eee-convert fix	141	0	1
ecd1db15	docs(e2e): add NOTES.md for the venv pin + from-spec conversion gotchas	54	0	2
1b75bd0c	docs(planning): olmo from-run-spec migration plan	252	0	1
5c31a05f	feat(olmo-from-spec): Change 4 discovery dry-check + measured baseline	347	30	3
b2ebad72	feat(olmo-from-spec): Change 1 – from-spec preset fields + run-entry reconciliation	180	79	6
ab5bdc2e	docs(planning): handoff prompt to continue the olmo from-spec migration	143	0	1
99bdc0ee	feat(olmo-from-spec): Change 3 – wire –from-spec into the smoke/full grids	20	2	2
037ba682	test(olmo-from-spec): Change 6 – discovery dry-check + comparability proof	174	0	1
1ad68a8b	fix(olmo infer-stack): Change 7 – resolve data_dir as env > settings.yaml pin > /data default	61	7	2
90d95811	docs(olmo-from-spec): Change 8 – README from-spec default + annotate NOTES as resolved	145	43	9
91413bac	docs(olmo-from-spec): mark Changes 2/3/6/7/8 done; only Change 5 (GPU) remains	148	49	3
5accb6cd	fix(olmo container): install eval_audit in runner image so in-container helm-run loads the olmo-7b tokenizer override	71	1	2
536b0607	docs(journal): in-container olmo-7b tokenizer override (eval_audit missing from runner image)	23	0	1
2e1c40e3	docs(planning): multi-model from-spec via local-deployment-scoped discovery strip	233	0	1

2026-06-30 (29 commits)

Hash	Subject	+	-	Files
ae14bc77	fix(container): deliver HF token via mounted cache, not env-forwarding	81	0	2
ebcc5b7d	docs(container): correct HF-token delivery mechanism	49	14	6
c763a6ea	docs(journal): HF token on-disk delivery for containerized runs	24	0	1
9ce64278	fix(olmo container): install crfm-helm[all] + pin huggingface_hub==0.36.2	25	10	3

Hash	Subject	+	-	Files
0fdc7457	test(olmo smoke): preflight probes container python env (langdetect + hf_hub pin)	57	5	2
528d341a	docs(journal): container env fix (crfm-helm[all] + hf_hub pin) + smoke probe	25	0	1
bf2b9767	test(olmo smoke): add wmt_14 + ifeval recipe canaries to smoke grids	23	3	2
676746d0	docs(journal): wmt_14 + ifeval smoke canaries (from-spec-constrained selection)	16	0	1
99e27c51	test(e2e): preflight probes container python env (langdetect + hf_hub pin)	53	3	2
3b32c1eb	docs(journal): e2e container-env probe (no canary, per request)	14	0	1
8d422490	docs(journal): resolve OLMo-canary question (kept; "no canary" was e2e-only)	1	1	1
8dd861b5	docs(planning): address reproductions by (public_root, relative_path)	369	0	1
b7b115cc	docs(planning): rel-path plan – kwdagger cross-products, broadcast map mandatory	15	17	1
e94d2c86	docs(planning): rel-path plan – host-side resolution + kwdagger submatrix	284	300	1
1d1097d0	docs(planning): rel-path plan – materialize substituted copies host-side	135	92	1
77f13574	test(kwdagger): pin the submatrix-zip contract the rel-path plan relies on	123	0	2
515bddbd	feat(from-spec): host-side run_spec resolver + materializer (rel-path plan §4.1)	452	0	2
6305d294	feat(from-spec): wire exact-path replay through the bridge + docker node (§4.2-4.6)	443	54	4
22026464	test(from-spec): pin run_spec_json on the from-spec node algo_params	6	3	1
337ac8d4	feat(from-spec): make-manifest –run-spec-sources-fpath for exact-path replay (§4.5)	157	4	2
c8fe9b76	docs(from-spec): mark rel-path plan partially implemented + journal entry	24	1	2
bbbeaeb7	fix(from-spec): gate materialized per-run lease_endpoint on –lease	25	3	2
5f568bf5	feat(from-spec): exporter auto-freeze of rel-paths (rel-path plan §4.5)	287	4	3
a9a1a16b	feat(from-spec): dry-check existence mode for frozen manifests (rel-path plan §4.7)	202	0	2
249735d3	docs(from-spec): mark rel-path plan host-side-complete + journal entry	24	10	2
cea323f2	fix(from-spec): content-address the materialized run_spec filename (kwdagger identity)	69	11	3
d9b60ec8	docs(from-spec): document kwdagger caching model + content-addressed identity	35	5	2
115a9997	debug(olmo): MWE deployment matrix for OLMo-7B fp16-collapse diagnosis	1170	0	9
74ba33de	fix(olmo-7b): serve OLMo-1.7 tokenizer to stop spurious EOS append	16	0	1

2026-07-01 (15 commits)

Hash	Subject	+	-	Files
ab9ccce8	feat(deployment-match): general HELM deployment-match search tool (Phase 1)	1687	0	11
86ad9598	feat(deployment-match): Phase 2 – two-tier serve+probe driver (‘run‘)	169	0	2

Hash	Subject	+	-	Files
aee205b4	feat(deployment-match): Phase 3 – confirm winner via compare-pair (‘confirm‘)	174	0	2
792ce211	test(deployment-match): Phase 4 – pytest coverage + testable tokenizer predicate	196	14	2
c978dea6	docs(deployment-match): mark plan IMPLEMENTED + document run/confirm	60	16	3
88667c69	tooling(catalog): catalog core-report packets by decoding temperature	234	0	1
d703ad4c	feat(deployment-match): ‘auto‘ – one-shot dry-run->run->score->confirm	73	1	2
c8d8417d	fix(deployment-match): let infer-stack place GPUs (acquire –queue), don’t pin	38	9	4
aaab9a35	tooling(catalog): detect chain-of-thought alongside temperature	143	54	1
f04edd85	fix(from-spec): thread freeze_rel_paths through export->materialize	54	3	3
b998f53f	feat(olmo): combined multi-model preset for parent-root fan-out	209	7	2
bb2d959d	feat(olmo): combined multi-model fan-out runbook	613	0	13
c6d7a294	chore(olmo): remove superseded fp16-collapse debug MWE	0	1154	8
d6534170	fix(from-spec): strip model_deployment token from exact-path locator run_entry	67	1	2
a2879335	chore(olmo): remove default GPU restriction (serve across all detected GPUs)	12	7	3

2026-07-02 (6 commits)

Hash	Subject	+	-	Files
88262878	fix(olmo): enable public comparison in combined vexp (was local-only)	12	12	1
065b683a	docs(olmo): remove stale "local-only by default" claims	17	18	3
a14f2faa	docs(planning): plan for from-spec adapter_spec decode-param overrides	158	0	1
71249b9d	Merge branch ’jons/fable-commits’ into impl/run-from-run-spec	0	0	0
3014cea3	docs(journal): session entry for fable-commits review + merge	12	0	1
52513fc9	feat(olmo): fold base olmo-7b into the combined fan-out vexp (all six models)	127	38	8

2026-07-03 (34 commits)

Hash	Subject	+	-	Files
a90a084f	docs(planning): full codebase audit – findings catalog + phased fix plan	587	0	2
63cac3fc	test(kwdagger): guard collection with importorskip("kwdagger") (P0-9)	13	0	5
fa8b745a	fix(cli): repair analyze_backlog import + canonical report layout (P0-8)	9	8	2
a5a2a2f5	build(deps): pin transformers<5 / hf-hub, declare direct deps (P1-24)	13	2	2
3e98614f	fix(summary): tol010 sankeys bucket on the 0.01 curve point (P0-1)	68	8	3
6d911222	fix(index): resolve unstamped local index first; single-source resolver (P0-2, R-6)	108	22	4
dbe3b68f	fix(planning,virtual): match/join on canonical keys, not raw strings (P0-3, P0-4)	171	7	5
05c9dd62	fix(helm): correct perturbed/unperturbed agreement split (P0-6)	97	4	3
b42064c0	fix(planning): unknown-on-partial facts; extract official max_eval_instances (P1-1, P1-2)	134	6	4

Hash	Subject	+	-	Files
09fc3294	fix(normalized): filter non-finite scores at row construction (P1-15)	181	33	5
d05126d0	fix(reports): cross-machine curve uses rel_tol=0 highlights (P1-13)	93	1	3
0201ec06	fix(eee): derive experiment name from local/<experiment>/ layout (D-1)	59	28	7
86d41ae5	fix(bundle): content-address model_deployments.yaml filename (P0-5)	52	3	2
5a7d270d	fix(pipeline): precomputed_root is algo identity on from-spec node (P1-21)	30	5	2
2896be59	fix(manifests): missing-entries NameError; reject duplicate labels (P1-22, P1-23)	25	2	3
54bf35f4	fix(docker): image-id identity for unpinned images; –name + teardown (P2)	144	13	6
c5ac0203	fix(summary): failure-taxonomy honesty – colors, categories, ordering (P0-7, P1-8, P1-11)	152	11	4
7642a1b6	fix(summary): not_analyzed status level; surface unreadable bundles (P1-9, P1-10)	87	19	4
c5d7cdb5	fix(filter): one sankey row per excluded run, primary reason (P1-12)	38	2	2
cdac08ec	fix(eee): status.json gate on cached/local resolvers; unconditional stale-scope delete (P1-7, P1-6)	94	13	4
6dac5e99	refactor: delete the no-op –allow-single-repeat flag (D-3)	16	32	6
ffac9b4e	fix(cli): analyze-many exits non-zero on failure; warns on incomplete summary (P1-19)	74	0	2
41e5b962	fix(core-metrics): –no-plots preserves previously rendered figures (P1-16)	26	7	2
2537b388	fix(summary): thread invocation flags into reproduce.sh (P1-20)	72	5	3
b3130f42	fix(stale-state): prune stale packet + breakdown dirs on re-run (P1-4, P1-5)	37	1	2
0ae927fb	fix(determinism): stable benchmark discovery + HelmRunDiff tie-breaks (P1-17, P1-18)	57	8	4
a9a20979	fix(determinism): sort remaining nondeterministic outputs (P2, IM-12)	28	5	4
bdf94a70	fix(cli): –dry-run vs –run 1 errors; correct stale argv test (P2)	12	1	2
688831a3	fix(helm-diff): f-string prefix + None-safe group sort (P2)	7	3	1
14ce8042	fix(cli,loaders): drop debug blocks; count skipped samples lines (P2, IM-3)	13	13	2
99396f3f	fix(secrets,coverage): close HF-token perms window; return sankey paths (P2, IM-9)	14	2	2
235b555e	fix(loaders): tolerate non-numeric timestamp; preserve EEE provenance (P2)	14	5	1
10c8f32d	docs(journal): session entry for Phases A–F audit-fix implementation	22	0	1
9303ec5e	docs(preset,runbook): representative-subset wording (D-2); hashed bundle filename in tree listing	7	5	2

2026-07-06 (50 commits)

Hash	Subject	+	-	Files
0608b29c	test(planner): update stale slow-suite assertions to canonical-key behavior	15	9	1
15487f8e	fix(secrets): chmod 0600 the model_deployments bundle YAML holding the live key	24	0	2
6da780de	fix(reports): route Stage-6 writers through atomic fs_publish helpers (IM-1)	18	14	3
607189c8	fix(reports,cli): surface mpl fallback error; fix eee caveats indentation	48	7	3
b127aa48	fix(cli,adapter): fail fast on backlog/preset/base_url misconfig (P2)	179	12	7

Hash	Subject	+	-	Files
20a16f5f	fix(workflows): dedupe kwdg re-attempts, bucket load errors, preserve CSV dtypes (P2)	151	14	3
ed576083	fix(reports,cli,loaders): honesty fixes IM-6/IM-8/IM-5	208	3	7
0e324ed1	perf(reports,loaders): behavior-preserving IM-2/IM-7/IM-4	33	28	4
e9cf6c44	build(deps): bump python floor to 3.12; annotate aiq-magnet dep pins; relock	537	1318	2
dc8f680e	test(eee): centralize EEE-demo fixture path + skip guard in conftest (IM-11)	70	30	7
e09c9176	docs(eee-vs-helm): document non-comparable run-level agreement definitions (IM-13)	34	0	1
8a9933c7	docs(journal): session entry for Tier-1/Tier-2 audit-backlog implementation	30	0	1
ef6903fb	fix(secrets): warn on stderr when bundle key perms cannot be tightened	9	0	1
bab7df2c	refactor(reports): retire reports/quantiles.py (D-4)	2	193	3
e0d51e6b	feat(manifests): –max-eval-instances official sentinel for verbatim replay (D-5)	136	4	5
5b1c69f2	refactor(indexing): single classify_model_eligibility policy (R-7)	211	94	3
833a84bb	refactor(summary): single _classify_filter_gates; delete dead sankey builders (R-4a/R-4b)	85	392	4
99ff4494	feat(reproduce/olmo): deployment-match runbook for olmoe-1b-7b/ifeval	130	0	1
c8cffa42	fix(summary): align the two filter-funnel families on metadata + judge gates (R-4c)	85	1	5
fe7fe271	refactor(summary): rename ambiguous agreement-tolerance key family (R-4d)	69	58	8
b6bcd245	refactor(normalized): dedupe EEE conversion core + one aggregate predicate (R-5)	185	165	5
ba730045	refactor(normalized): one metric_handle helper; drop dead members (R-9)	21	58	5
899e0fd3	refactor(core-metrics): dead-surface sweep + assert-membership tol guard (R-10)	41	254	7
143d3fac	refactor: misc dead-code sweep (R-11)	70	150	10
e5c1bdb4	refactor: one run-entry model_deployment token utility (R-8)	165	34	5
7e1f5f12	refactor(infer_stack): extract PRESET_CONFIGS to presets.py (R-3, pure relocation)	1162	1140	2
d895a446	refactor(infer_stack): promote discovery core to public discovery.py (R-3, R-3d)	105	78	4
e7eab858	refactor(infer_stack): extract serving_facts.py (R-3b, pure relocation)	165	128	2
9a808120	refactor(infer_stack): extract freeze.py (R-3d, pure relocation)	93	76	2
ac5ca26a	refactor(infer_stack): extract bundle_export.py; adapter is now a facade (R-3c)	667	642	3
79bc030c	docs(journal): session entry for Tier-3 structural refactors (D-4/D-5, R-3..R-11)	31	0	1
560cdda0	docs(planning): stamp audit doc IMPLEMENTED (except R-2) with pass summary	17	0	1
cfe7d6a6	docs(planning): simplicity & bloat audit findings + phased implementation plan	147	0	1
5b7b62bb	chore(dev): delete dead POC and oneoff trees; fix docs that referenced them	22	6901	25
1457a269	chore(configs): drop checked-in generated manifests and dead examples/	0	1470	25
8b4c69d5	refactor: remove helm/hashers shim, dead agreement profile, if-1 block	13	178	8
90af80ba	docs: fix first-run and accuracy issues in README, reproduce index, setup	79	82	3

Hash	Subject	+	-	Files
77e126b4	chore(docs): move paper analysis scripts out of docs/, archive session logs	32	17	9
425e05a2	docs(planning): archive 12 implemented plans; consolidate olmo/infer-stack clusters	193	132	67
24390f26	chore(journals): rotate 544K of append-only journals to archive	5542	5526	4
c804cdc1	refactor(utils): finish R-6 helper consolidation into utils/coercion	159	137	14
32622e0b	refactor(reproduce): merge three divergent _lib.sh copies into one shared lib	437	374	4
51797aa9	refactor(workflows): split _render_scope_summary into per-stage render helpers	458	197	1
a07736c2	refactor(reports): split core_metrics.main into per-section helpers	182	57	1
3bd43bf5	refactor(infer_stack): move PRESET_CONFIGS catalog to a YAML resource	965	1009	3
a849ea32	refactor(reports)!: migrate pair_report/pair_samples onto Normalized-Diff (R-2 stage A)	284	117	4
a0cadf26	refactor(helm)!: delete the legacy agreement half of diff.py (R-2 stage B)	59	986	6
a34241ca	docs: stamp simplicity audit IMPLEMENTED; journal entry for the session	69	2	2
066cc520	refactor(reproduce/olmo): self-contained combined runbook; remove stale single-run runbook	323	1655	24
7ba01845	chore: remove configs/docs orphaned by the single-model OLMo runbook removal	28	481	13

2026-07-07 (22 commits)

Hash	Subject	+	-	Files
a1f893fa	fix(deployment_match): clamp max_model_len to the model's derived ceiling	85	9	4
f4241a87	docs(deployment_match): vLLM-vs-HuggingFace deployment-match knob inventory	166	0	1
2a9fb007	feat(deployment_match): hf-match grid profile to match a HuggingFace-Client run	126	4	5
008987b1	docs(deployment_match): record full sampling replay as a deliberate non-goal	9	3	1
e4e4b7c7	feat(deployment_match): sweep/pin attention_backend via the infer-stack option	109	35	6
d5cb46e8	docs: instruct agents to proactively commit logical units of work	11	0	1
acf6ca44	docs(deployment_match): correct the confirm-catalog fidelity gap description	15	6	1
0a755c73	fix(deployment_match): confirm catalog reproduces the winning serving runtime	69	18	5
070f986c	feat(deployment_match): sweep attention_backend in hf-match instead of pinning	55	36	4
7a0908ea	feat(reproduce/olmo): default the OLMoE deployment-match to hf-match	18	10	1
3fd80ddb	fix(deployment_match): raise max_num_batched_tokens >= max_model_len for hf-match	46	2	2
4b8eb37d	chore(submodules): bump cmd_queue to dev/0.3.2, kwdagger to dev/0.2.6	4	4	3

Hash	Subject	+	-	Files
c436308b	chore(submodules): bump infer_stack to latest infer-stack-cli-api-migration	1	1	1
2640e108	feat(deployment_match): endpoint/cell counters + ETA in the serve loop	41	9	1
f855dea9	feat(deployment_match): –dtypes/–attention-backends knobs; OLMoE skips fp32	83	0	4
608bb45f	feat(deployment_match): preflight feasibility filter (generalizes fp32-MoE prune)	151	27	6
a8dd792b	fix(deployment_match): hf-match pins protocol=completions (HuggingFaceClient repro)	36	1	3
fbc2040d	fix(deployment_match): revert wrong completions pin; HuggingFaceClient IS a chat client	55	31	3
1fb91576	feat(deployment_match): –log-requests / –extra-serve-args to expose the vLLM prompt	73	0	5
d32abd3b	feat(deployment_match): compare_prompt.py – diff HELM get_prompt vs vLLM /tokenize	266	5	4
b3ae0168	feat(deployment_match): sweep add_generation_prompt (OLMoE chat-template version drift)	96	12	5
19c0ec1c	bump infer_stack pointer	1	1	1

2026-07-08 (26 commits)

Hash	Subject	+	-	Files
1690b7ac	feat(deployment_match): per-model ifeval runbook scripts for the 3 OLMo-2 instruct models	495	0	3
c2c3978d	Merge branch 'impl/simplicity-audit' into integrate/simplicity-audit	0	0	0
37a411a5	docs: fix dangling markdown links after the simplicity-audit merge	81	81	9
9059dad2	Merge branch 'integrate/simplicity-audit' into impl/run-from-run-spec	0	0	0
2698389c	feat(deployment_match): –fp32-tensor-parallel-size to serve fp32 MoE on N GPUs	151	11	6
426710fd	docs: inventory HELM's unrecorded deployment params + HF-revision pinning effort	231	0	2
a8c573a3	docs: use precise terms for the provenance machinery	47	5	1
37d91aff	feat(deployment_match): hf-probe – reproduce the official at fp32 via HF and score vs oracle	401	12	4
f52afee7	docs: confirm OLMoE fp32 exact-match + scope the dtype gap to all OLMo-2 HF runs	35	2	2
79ab77b6	feat(deployment_match): DM_HF_FP32 + DM_FP32_TP on the OLMo-2 7B/13B/32B runbooks	127	33	3
2d792012	feat(hf-inprocess): reproduce HuggingFace-deployed officials on reserved GPUs	765	18	6
19cf7d40	bump infer-stack submodule pointer	1	1	1
9773233e	Add plan to allow reproductions of old HELM runs in docker containers with package versions pinned to the ones used during the era	90	0	1
545357e7	feat(heatmap): aggregate-score-diff heatmap (color = local-public, cells annotated with both scores)	631	0	4
59587f29	docs(journal): aggregate-score-diff heatmap session	61	0	1
c8707ead	feat(summary): auto-emit aggregate-score-diff heatmaps in build_reports_summary	234	62	5

Hash	Subject	+	-	Files
f87e92ea	docs(journal): auto-emit aggregate-score-diff in build_reports_summary	48	0	1
ce63eb40	docs: mark HF-in-process plan implemented (routing pending) + journal entry	77	2	2
5b352269	docs(deployment-match): point to the built HF-inprocess resolver + mechanism	19	0	1
0c8d0eb2	feat(summary): spell out agreement-bucket thresholds on plot legends	93	12	4
0846fb99	docs(journal): agreement-bucket threshold labels on legends	50	0	1
564b087a	Bump infer_stack pointer	1	1	1
c00394ee	feat(deployment_match): overnight batch – HF fp32 + vLLM sweep for all OLMo instruct models	124	0	1
3ff73c5c	feat(deployment_match): hf-probe sweeps dtypes (–dtype comma list) + DM_HF_DTYPES	91	39	8
70c867da	feat(heatmap): holistic headline-metric aggregate-score-diff heatmap	376	10	7
4c2de25b	docs(journal): holistic headline-metric aggregate-score-diff heatmap	55	0	1

2026-07-10 (16 commits)

Hash	Subject	+	-	Files
29fc76c4	feat(heatmap): headline plot uses modelsxbenchmarks layout + squared error	116	56	3
236b3289	docs(journal): headline plot layout + squared-error encoding	40	0	1
50a7117b	docs(journal): fp32 reproduces all OLMo instruct officials + HF-probe OLMo-2 fidelity bug	97	0	1
4516b704	feat(hf-probe): sweep forward-pass numerics (attn_impl/device_map/decode) to reproduce OLMo-2 officials	225	54	5
807144de	docs(journal): OLMo-2 HF divergence is a fp32 forward-pass diff + hf-probe numerics-sweep axes	118	0	1
d6cc37fd	feat(overnight): sweep every HF + vLLM parameter across all OLMo models	107	7	5
60921441	docs(journal): overnight full-sweep follow-up (HF+vLLM axes across all OLMo models)	23	0	1
25b00db3	feat(eras): era-pinned HELM registry + resolver (commit 1/6)	475	0	5
836bbb52	feat(eras): era image build path in build.sh + era dockerfile (commit 2/6)	507	25	4
8d54db18	feat(eras): helm_era_shim replay CLI + OpenAI-compatible client (commit 3/6)	834	0	5
3670050f	feat(eras): host-side era yaml + materializer guard (commit 4/6)	252	19	5
38940ff2	feat(eras): thread era through manifest + pipeline + bridge (commit 5/6)	379	1	6
a520062b	docs(eras): runbook + docs + journal (commit 6/6)	358	1	9
5f17e3e3	docs(eras): multi-angle code-review findings for the era implementation	479	0	1
11fa971d	feat(eras): portable validation-ladder harness + static era-import checker	738	5	10
39c020b2	docs(era-tests): plan the pre-v0.5 gates' migration to a turnkey dev runbook	431	0	2

2026-07-11 (32 commits)

Hash	Subject	+	-	Files
0c58ee96	fix(era-shim): v0.2.4 compatibility + credentials + run-dir safety (findings 1,2,3,7,8)	300	49	4
befd490d	fix(era-export): api_key in args, explicit base_url, multi-endpoint lease (findings 2,5,10)	133	14	4
f549943e	fix(era-guard): digest-pinned image era<->image guard (finding 4)	107	16	3
9bf2228b	fix(era-image): persist era identity into runtime ENV (finding 6)	14	0	1
f09051be	fix(eras): fail loud on track-less era-suite paths (finding 9)	48	2	2
0486bb79	refactor(era): process-context timing, session reuse, setdefault (below-the-cap)	53	2	4
4bba8dfa	docs(era): record findings resolution + deferred cleanups	18	0	1
67c0b53f	feat(era-tests): checked-in serving config + era presets + vexp manifests	192	0	5
99ff4dfd	feat(era-tests): dev runbook mirroring dev/e2e-tests (moves the pre-v0.5 gates)	959	429	24
3046c8ea	docs(journal): full era-path remediation + dev/era-tests runbook	71	0	1
4c181be7	fix(era-tests): review fixes – v0.2.4 master-key path + rung mirror-root join	117	11	8
981dfa1f	feat(era-tests): swap subject pythia-6.9b -> redpajama-3b to fit an 8GB GPU	204	111	13
c2598b49	fix(era-build): import run_benchmarking from helm.benchmark.run, not .runner	22	2	2
2e8338ff	fix(era-build): pin pyarrow==11.0.0 in era constraints (datasets 2.5.2 compat)	10	0	2
4dc3c4a4	fix(era-build): adopt era frozen requirements.txt as constraints (enriched seed)	451	23	3
16ee1eeb	fix(era-build): revert enriched freeze; setup.cfg ~= ranges already era-pin	47	425	3
e6a21ed1	docs(era-build): warn against re-adopting the stale era requirements.txt	16	0	2
37677132	fix(era-gate): rung-2 diffs display_requests.json, not the unshipped scenario_state.json	108	35	3
cb14ec80	fix(era-gate): sanitize colons from rung-2 dry-run mount path	5	1	1
1c552be2	fix(era-build): install wget + unzip in the era image for scenario downloads	9	1	1
38260572	feat(reproduce): from-spec gpt-oss-20b runbook (bbq, ifeval, mmlu_pro, gpqa)	1006	0	19
e40ee63d	fix(era-build): pin fsspec==2022.8.2 (datasets 2.5.2 LocalFileSystem skew)	14	0	2
6b23e9d8	fix(era-gate): forward HF token into rung-2 dry-run (script-dataset auth)	4	0	1
d6172b1e	docs(planning): qwen text-family combined fan-out implementation plan	377	0	1
973b5b7c	feat(qwen-fanout): qwen-combined preset + 8 from-spec members (generalize combined builder)	1395	92	4
fcd14973	feat(reproduce): qwen_models_combined runbook + infer-stack catalog + virtual experiment	1111	0	15
531bc4f8	docs(planning): mark qwen fan-out plan implemented (T1 confirmed; V2/V3/V5/V6 deferred)	17	1	1
47200f34	docs(journal): qwen combined fan-out implementation session	61	0	1
7620c246	fix(era-gate): rung-2 skips unfetchable-data probes as environment filters	32	0	2
629c4541	docs(planning): plan for producing NEW Qwen 3.6 core results (compute, not reproduce)	339	0	1
faef9ea4	docs(planning): rewrite Qwen 3.6 new-results plan around leased fan-out	212	253	1
17f02583	fix(provenance): double-quote the label key in image_label's Go template	24	1	2

2026-07-12 (39 commits)

Hash	Subject	+	-	Files
eb1b9faf	Merge impl/era-pinned-helm-containers into impl/run-from-run-spec	0	0	0
34929ae4	docs(journal): era-pinned -> run-from-run-spec merge session	16	0	1
ea74ac38	docs(planning): repo simplification/refactor plan (4th pass, post-era)	479	0	1
bffb84a6	docs(planning): second-pass review revisions to the simplification plan	141	96	1
dee2ba95	docs(journal): fourth simplification-pass audit + plan review session	61	0	1
b77bdf82	docs(planning): A4 gate caveat – committed phase3 baseline is EEE-only	6	1	1
bf9172f4	docs(planning): live-code read pass – deltas R-g..R-m + new item E5	100	15	1
911a993f	refactor(E1): delete five grep-verified dead symbols	3	135	6
f59060cb	refactor(E2): de-duplicate the copied plotting helpers	97	78	8
b7300a19	refactor(E4a): prune facade re-exports with zero internal+external refs	20	78	3
cc487686	refactor(E5a): retire the deprecated EVAL_AUDIT_EEE_STRICT env override	19	19	2
88c6e389	refactor(B3): era config/build dedup – single source of truth	55	36	5
96b88300	chore(F2): trim vestigial norecursedirs entries	5	10	1
4dae513c	refactor(D1): shared index-resolution + reproduce-script helpers	111	88	5
7926209e	refactor(D2): shared EEE render helpers; single-file meta extractor	150	104	4
1f06e9a5	refactor(A1): move run_entries out of helm/ to eval_audit/run_entries.py	46	23	20
9f7c6abd	refactor(B1): one shared benchmark_output path parser	50	30	3
7e8b527d	fix(E4a): restore _single_run_core_stat_index facade re-export	1	60	3
8078cc1f	refactor(A2): delete the prod-dead half of helm/	42	655	8
eb313c08	refactor(A3): one json_compatible implementation (utils/jsonify.py)	67	60	3
09449188	refactor(C1): finish the build_reports_summary split (1,715 -> ~330)	1550	1397	6
364925b6	refactor(B2): single named era_mode flag in the bundle exporter	18	5	2
648193d3	refactor(D4): portfolio_status logic -> reports/portfolio.py	414	390	2
4d4085ef	refactor(E3): local artifact writers in the filter-report emitters	71	45	1
3d077a69	refactor(C3): move the repair/publish half out of breakdown.py	214	203	4
e630a576	refactor(D3): layer the failure classifiers – one generic taxonomy	23	2	1
c42e8159	docs(E5b): –skip-diagnosis kept – phase3 4.8 closed as permanent	11	0	2
c9d6c38d	refactor(C2): extract the shared heatmap grid scaffold	212	221	1
950ae99b	docs(planning+journal): mark plan Batches 1-3 implemented	81	2	2
36bb74e0	refactor(D5): deprecate compare_batch (phase 1, no behavior change)	29	0	3
e1b8547d	test(A4): extend phase3 baseline capture to the HELM render path (F1/F2)	2952	18	5
4638f788	docs(planning): record 2026-07-12 operator decisions + session journal	106	0	2
75366f42	chore(review): two lint fixes from the subagent-work review	3	3	2
e295c13b	fix(era): set tokenizer_name + max_sequence_length on the era model deployment	78	11	3
a3a33010	fix(era-build): pin nltk==3.7 + provision punkt so BiasMetric tokenizes	23	0	3
3651eeb1	fix(runbooks): drop the deleted –allow-single-repeat flag from all callers	9	13	8
3a9aa30b	feat(presets): merge runbook-local presets via INFER_STACK_EXTRA_PRESET_FILES	96	0	2
504c6b9f	feat(reproduce): classic Together open-weight combined era-pinned runbook	2546	0	17
b344671d	fix(era-shim): render credentials.conf from the baked deployments key, not the env	74	2	2

2026-07-13 (16 commits)

Hash	Subject	+	-	Files
47658cf7	fix(olmo): pass a placeholder –base-url in the freeze-only preflight	9	1	1
81c71002	fix(gpt-oss,qwen): placeholder –base-url in the freeze-only preflights	18	1	2
bb0a377b	docs(era): note classic officials are carried forward v0.2.4<->v0.3.0	18	0	2
339748af	docs(era): record original execution eras (v0.2.2 / v0.2.3), not the pinned eras	13	4	2
5000b8ac	docs(era): correct origin – pre-v0.2.2 (original HELM), not v0.2.2	19	12	2
b445a7fd	docs(planning): LiteLLM route-registry plan for the shared-gateway 400 bug	578	0	2
d5c70c94	fix(freeze): resolve subset run-entries to their exact dir, not AMBIGUOUS	76	5	2
f64c20bd	docs(planning): mark litellm-route-registry plan implemented	6	2	1
1fbf4cbf	docs(journal): route-registry implementation session	70	0	1
9ff068c8	feat(classic-together): run all targets in parallel, lease arbitrates GPUs	168	167	4
27e4ee40	docs(era): document classic-corpus class_name migration + true origin window	163	12	3
c44a6467	chore(submodule): bump infer_stack to route-registry merge (22d94317)	1	1	1
c3e68bed	feat(era-shim): canonicalize relocated class paths (G13 fix)	214	3	3
3d904b92	fix(gpt-oss): null-safe chat clients normalize content:null -> "" (HELM untouched)	250	18	6
32657a62	fix(docker): bump runner image Python 3.11 -> 3.12 to match eval_audit floor	9	5	3
c9f0efac	fix(docker): prune selenium platform binaries / dangling symlinks before venv COPY	13	0	1

2026-07-14 (15 commits)

Hash	Subject	+	-	Files
8d0d2c4b	feat(reports): instance-level fallback for aggregate score-drift plot	351	42	8
f34d9f96	docs(journal): instance-level fallback for aggregate score-drift plot	52	0	1
c041abb0	fix(heatmap): headline metric resolves over metric family + main split	178	10	2
b83c03e9	docs(journal): headline metric family/main-split resolver fix	43	0	1
33864be3	Make eval audit run kwdagger with other_session_handler=kill	1	0	1
ca885180	fix(reports): align coverage-matrix axis order with the score-drift plot	86	11	3
cdb31422	feat(reproduce): add 50_rsync_from_aiq_gpu.sh to olmo/gpt-oss/qwen runbooks	360	0	6
d4bf29d7	feat(coverage-matrix): show match stats per column + align model order	159	3	2
01351b73	docs(journal): coverage-matrix annotations + axis alignment	43	0	1
7a9f1d5f	feat(coverage-matrix): print agreement proportion in each cell	120	118	2
93849b09	fix(heatmap): dedupe stale local attempts in aggregate score-drift	175	1	2
bec1f57c	docs(journal): aggregate score-drift stale-local dedupe	57	0	1
e1d10217	test(e2e): route-registry survival acceptance check	201	0	2
a36b787c	fix(heatmap): keep headline drift cells square for small model counts	73	1	2
b8b85869	fix(qwen): serve 8192 window on the instruct-turbo endpoints for from-spec ifeval/mmlu_pro	73	9	3

Appendix I

Submodule contribution ledger

This appendix records Edward-authored commits in the included submodule histories during the internship window. Merge commits can have zero numstat even when they integrated substantial branch work.

`aiq-magnet` (3 commits)

Date	Hash	Subject	+	-
2026-05-29	`762ca6b5`	Make run matching more robust	129	13
2026-06-25	`4b10e1ba`	helm: add materialize_helm_run_from_spec (faithful run_spec.json replay)	931	0
2026-06-25	`50489f0a`	from-spec: optional –model-deployment rewrite (record the local endpoint)	208	25

`cmd_queue` (1 commits)

Date	Hash	Subject	+	-
2026-06-23	`0cb1ac21`	Add first-class job setup/teardown lifecycle	107	4

`infer_stack` (30 commits)

Date	Hash	Subject	+	-
2026-05-20	`328cd1c4`	Add plan for lora implementation	151	0
2026-05-20	`6af064a0`	Merge pull request #2 from AIQ-Kitware/lora_branch	151	0
2026-05-27	`538e4c5c`	Expose ports in the .env file	32	5
2026-06-23	`8150e331`	Fix TUI resize bug and collapse serve and acquire + other stuff	209	175

Date	Hash	Subject	+	-
2026-06-23	79902140	Add admission queue to acquire (wait_for_placement / –queue)	223	3
2026-06-23	55c39a78	Add ‘infer-stack gc‘ to reclaim leaked leases and free GPUs	145	1
2026-06-23	156f6bb7	Render LiteLLM with a static superset route table (no-blip gateway)	230	24
2026-06-23	af4d1d6e	Merge feature/litellm-no-blip into the leasing pipeline lifecycle	230	24
2026-06-23	bfb7dbe7	Merge feature/leasing-and-no-blip into dev/leasing-controller	597	27
2026-06-23	cfddfac0	Merge dev/leasing-controller: pipeline leasing lifecycle + no-blip gateway	22700	12364
2026-06-24	4381731e	leasing: tolerate configs without a ’catalog’ field in _load_catalog	6	2
2026-06-24	995f45e1	Merge dev/lease-polish: leasing/e2e polish + compose-backend catalog fallback	524	34
2026-06-25	0cde11d5	Merge dev/lease-polish: protocol-aware generation-gated readiness + –catalog routing for release/gc/evict	226	49
2026-06-25	933c3f2c	gitignore: ignore setuptools build/ output dir	1	0
2026-06-29	045536cb	Merge dev/lease-polish: host-port churn fix, reconcile locking, dynamic routing	2708	162
2026-06-30	e2f51fdb	Merge remote-tracking branch ’origin/dev/lease-polish’ into infer-stack-cli-api-migration	375	71
2026-06-30	104d4254	leasing: release lease when a not-ready acquire times out	178	10
2026-06-30	beb1cc66	fix(leasing): refuse to mutate without a real cross-process lock; group-write new lock files	300	16
2026-06-30	7fa342dd	fix(tui): preserve scroll offset when a table rebuilds on poll	88	5
2026-07-01	caa67503	fix(tui): require a double-click to acquire an endpoint	180	16
2026-07-07	6c586568	feat(leasing): attention_backend endpoint option (VLLM_ATTENTION_BACKEND)	101	2
2026-07-07	d685573c	feat(tui): API tab respects endpoint completions vs chat protocol	122	16
2026-07-07	7f5fafed	fix(tui): handle string-form reclaim spec in endpoint edit screen	5	1
2026-07-08	e5fba7b5	feat(leasing): reserve-only GPU lease (acquire –reserve-gpus N)	339	14
2026-07-08	1109b0f0	test(leasing): e2e probe for the reserved-GPU index-frame assumption	150	0
2026-07-08	c119a62e	docs(changelog): note the reserve-only GPU lease (–reserve-gpus)	17	0
2026-07-13	2d832b61	feat(leasing): append-only LiteLLM route registry (static-superset)	783	37
2026-07-13	869080bc	feat(cli): ‘infer-stack routes‘ group + route-registry docs	522	18
2026-07-13	c005a887	fix(tui): surface HTTP error bodies in the API tab	63	2
2026-07-13	22d94317	Merge feat/litellm-route-registry into infer-stack-cli-api-migration	1368	57

`kwdagger` (1 commits)

Date	Hash	Subject	+	-
2026-06-23	`ce246bae`	Add ProcessNode setup/teardown resource lifecycle	150	0